Privacy-Preserving Split Learning for Federated LLM Fine-Tuning

Heng Jin Chaoyu Zhang Hexuan Yu Wenjing Lou Y. Thomas Hou
Virginia Tech, Arlington, VA, USA

arXiv:2609.09794v1 [cs.LG] 9 Sep 2026

***Abstract*—Fine-tuning large language models (LLMs) on domain-specific data is essential for downstream adaptation. In many deployments, a participant cannot hold the complete model locally. This happens because the model owner keeps the full model proprietary, or because the participant lacks sufficient compute resources. Split Learning (SL) addresses this by partitioning the model between the participant and a server so that only a small portion runs locally. When the underlying data is additionally distributed across multiple institutions with privacy requirements, Federated Learning (FL) further enables collaborative training across participants by sharing only model updates instead of raw data. In this combined setting, each client transmits intermediate activations to the server, and for LLM fine-tuning, this exchange poses an inherent privacy paradox. The autoregressive nature of LLMs causes the transmitted activations to leak the input, and existing perturbation-based defenses are fundamentally ineffective in this setting. We address this leakage through a learned obfuscate-and-recover scheme that protects participants' private datasets while still allowing an independently deployable model to be trained on the server side. Experiments demonstrate that our approach achieves strong privacy protection with modest utility loss and system overhead, making split-based federated LLM fine-tuning practically viable.**

I. Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities across a wide range of natural language processing tasks, covering both proprietary and open-weight models. Fine-tuning a pre-trained LLM on domain-specific data is essential for adapting it to downstream applications, with common paradigms including supervised instruction fine-tuning [1] and reinforcement learning from human feedback [2]. However, the data needed for such fine-tuning is often private and distributed. Hospitals hold sensitive patient records, law firms hold confidential case files, and enterprises hold proprietary business data. Centralizing this data on a single server raises serious privacy concerns and may violate regulatory requirements, making collaborative fine-tuning without data sharing a practical necessity.

Two largely independent constraints make this collaborative setting harder in practice. First, a participant may be unable to hold the complete LLM locally. This happens when the model owner is unwilling to give participants access to the full set of proprietary model weights, or when the participant's hardware cannot load or train a full LLM, a common situation for institutions with limited accelerator infrastructure and for edge devices such as smartphones or embedded systems. Split Learning (SL) [3] addresses this by partitioning the model between the participant and a server at a cut point, so that the participant executes only a lightweight portion locally while the server handles the bulk of the computation. Second, the fine-tuning data is often distributed across multiple institutions, each subject to its own privacy requirements. Federated Learning (FL) [4]–[7] addresses this by allowing each institution to train on its local data and share only model updates, so raw data remains local. These two constraints frequently co-occur in practice, since the resource-constrained institutions and edge devices that need SL are often the same participants holding the private data that motivates FL. Federated split learning [8]–[10] combines both techniques, letting each client execute only a lightweight portion of the model locally while still collaborating with other clients through federated aggregation.

However, applying SL to LLM fine-tuning poses a fundamental privacy challenge rooted in the autoregressive training objective. An LLM is trained to predict the next token given its input, so the input and label sequences are nearly identical, and leaking the labels is effectively equivalent to leaking the input. Assigning both the input-side and the output-side of the model to the client keeps raw inputs and labels local, yet SL still requires the client to transmit intermediate activations (smashed data) to the server. Because of the autoregressive nature of LLMs, a white-box server can reconstruct labels from these activations and thereby recover the original input. This leakage path makes state-of-the-art defenses that only perturb smashed data insufficient for LLM fine-tuning.

To address this challenge, we propose PrivPair, a framework that makes split-based federated LLM fine-tuning practical under our threat model. The key insight behind PrivPair is to deploy lightweight adapters on the client to obfuscate the smashed data sent to the server, so that these activations are no longer compatible with the server's decoder. After the smashed data is processed by the server and returned to the client, a recovery adapter maps it back to a representation compatible with the client-side model, preserving training utility. Figure 1 illustrates the resulting scenario.

Contributions. Our contributions are summarized as follows:

- We propose PrivPair, a client-side obfuscate-and-recover mechanism with two lightweight adapters that mitigates smashed data leakage while preserving trainability.
- PrivPair allows the server to retain a complete, directly deployable model after training, preserving the standard FL deployment workflow.
- We evaluate PrivPair against reconstruction attacks and baseline defenses, showing strong privacy protection, modest utility loss, and modest client-side computation

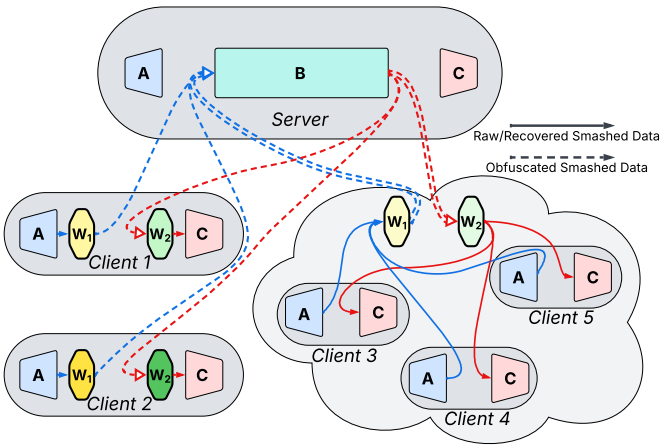


Fig. 1. Overview of the FL scenario with multiple clients and a server exchanging smashed data. Line colors are used only to improve visual distinguishability. Clients 3, 4, and 5 are mutually trusted and share a single adapter pair. The raw smashed data produced by A is obfuscated by W_1 before being sent to the server, and the server's response is recovered by W_2 before being consumed by C. After aggregation and global model synchronization, every client and the server hold the same A and C, while each client (or trust group) retains its own distinct adapter pair.

and memory overhead on devices with limited GPU memory.

Our artifact is available at https://github.com/hengvt/PrivPair_arXiv.

II. Related Work

A. Federated Learning and Split Learning

FL [4] and SL [3] represent two independent lines of work for collaborative model training. FL addresses data privacy by enabling multiple clients to collaboratively train a shared model while keeping raw data local, exchanging only model updates. SL addresses computational efficiency by partitioning a neural network at a cut point and offloading the major portion of the model to the server, so that resource-constrained clients need only execute a lightweight head locally. Federated split learning [11] combines both ideas. It adopts the model-partitioning approach of SL to reduce client burden together with the multi-client aggregation protocol of FL to enable collaborative training without sharing raw data. In LLM fine-tuning, the client may also retain the output-side component of the model to keep labels and loss computation local, which raises the risk of label leakage from the smashed data exchanged at the cut points.

B. Data Reconstruction Attacks

Despite the privacy-preserving objective of FL and SL, prior work has identified multiple attack surfaces, including property inference attacks [12], membership inference attacks [13], [14], and data reconstruction attacks (DRA) [15]–[18]. Among these, DRA is particularly concerning, as it directly targets the information exposed during collaborative training. If the server can reconstruct the client's raw training data from the information it receives, the central privacy motivation of FL is undermined. Existing DRA methods in SL target both the forward and backward communication paths. EIA [19] targets the smashed data transmitted from the client to the server. Given the intermediate hidden states produced by the client-side model, EIA trains an inverse mapper or performs optimization to find input tokens whose representations match the observed smashed data. DLG [20] initializes dummy inputs and labels, then minimizes the distance between their gradients and the observed gradients to recover the original training sample. TAG [21] extends DLG to Transformer-based language models with an additional regularization term, yielding higher text recovery rates. LAMP [22] further improves upon TAG by introducing a language model prior to guide the search toward natural text, combined with discrete token reordering to recover more accurate token sequences. BiSR [23] proposes a bidirectional attack that jointly exploits both the forward and backward paths, combining smashed data inversion with gradient matching to achieve stronger reconstruction in the LLM SL setting. This line of work motivates our focus on the smashed data and gradients exchanged at the split points.

BiSR also shows that label leakage remains severe even when both the input-side and the output-side of the model reside on the client. They attribute this persistence to two fundamental obstacles. The first is the autoregressive nature of LLMs. SL requires the client to transmit intermediate activations to the server, which can then exploit its white-box copy of the model to reconstruct the labels from these activations and thereby infer the original input. The second is the not-too-far property of LLM fine-tuning. Even if the client withholds its fine-tuned parameters from the server, the parameter differences introduced by fine-tuning are sufficiently small that the server can still use its copy of the pre-trained model to accurately predict the labels. Under the conventional FL objective of producing a complete, deployable model on the server, this second obstacle becomes even more severe, because the server can hold the same model as the clients and label inference becomes substantially easier.

C. Privacy Protection Methods

Existing defenses for SL include perturbation-based methods and training-time regularizers, both aiming to prevent the server from recovering the client's private input. Perturbation-based methods differ in where the noise is applied. Embedding ε-Privacy [24] applies noise at the embedding layer output, perturbing the token embeddings to satisfy a relaxed local differential privacy condition [25]. Smashed-Data DP [26]–[28] applies Laplace or Gaussian noise directly to the smashed data transmitted to the server. NoPeek [29] takes a training-time approach, adding a distance correlation regularization term to the training loss to minimize the statistical dependence between the smashed data and the original input. These methods share a common limitation in the LLM fine-tuning setting. The same representation sent to the server must both hide the private input and remain directly useful for the downstream model. This coupling forces privacy protection

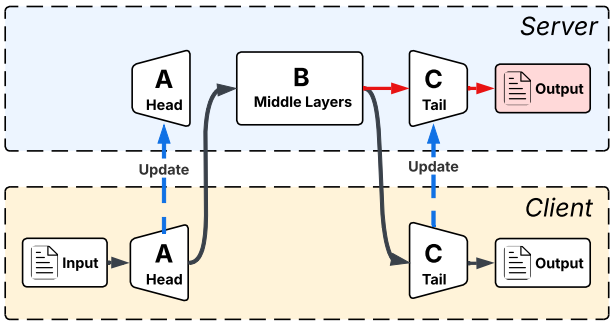


Fig. 2. The forward path of the baseline split FL system for LLM fine-tuning (one client shown as an example). The LLM is split into three components A, B, and C, with A and C residing on the client and B on the server. Note that since the server holds a copy of C, it can apply f_C directly to s_B to predict the labels, exactly as the client would.

and trainability to compete through a single smashed-data representation, motivating our obfuscate-and-recover design.

III. Threat Model

A. System Model

We consider a split FL system in which an LLM is partitioned into three components and distributed across clients and a server, as illustrated in Figure 2. Each client holds the head component A and the tail component C, while the server holds the middle component B. During fine-tuning, clients transmit the intermediate activations (smashed data) s_A to the server at the first cut point, receive the processed activations s_B back at the second cut point, and exchange the corresponding gradients in the backward pass. The server aggregates client updates via FedAvg at the end of each round.

Adversary's Capabilities. We assume the server acts as an *honest-but-curious* adversary. It follows the prescribed protocol and does not deviate from the training procedure, but it attempts to infer clients' private training data from all information it legitimately receives. The adversary has the following capabilities: (i) *Access to smashed data and gradients.* The server has full access to the smashed data and cut-point gradients exchanged by all clients at every training step. (ii) *White-box access to the model.* The server has complete white-box knowledge of the model architecture and current parameters, including both the server-side and the client-side portions. This knowledge excludes the client-side adapters introduced by our defense, which remain local to each client.

In conventional FL, clients typically perform multiple local training steps before uploading their updates, so the server has no knowledge of the intermediate client-side weights during those steps. To construct a stronger threat model, we grant the server knowledge of the client-side weights after every local training step. This ensures the server maintains full and current knowledge of the client-side model weights throughout the entire fine-tuning process without changing the training protocol itself.

Defender's Capabilities. The defender has the following capabilities: (i) *Arbitrary auxiliary dataset.* The defender may use an arbitrary text dataset that contains no private information. This dataset is independent of the private fine-tuning data and does not need to match its task, domain, or distribution. It may be obtained from any public or synthetic source. (ii) *Client-side adapters.* The defender may train two lightweight adapters on the client side. These adapters remain local at all times, are never disclosed to the server at any stage of training or deployment, and can be freely discarded once training is complete.

Objectives. The adversary aims to reconstruct as much of the original text from the client's private fine-tuning dataset as possible. The defender aims to prevent such reconstruction while ensuring that the fine-tuning process remains effective, producing a model of comparable quality to one trained without any privacy protection. The defender also requires the server to retain a complete, directly deployable model after training, consistent with the standard FL objective.

B. Scope

A separate class of attacks targets the model updates uploaded by clients at the end of each round. These gradient inversion attacks are independent of both SL and the LLM setting, and have been extensively studied in the standard FL literature, with well-established defenses such as secure aggregation [30], [31] and differential privacy [32]–[35]. This work does not address this attack surface. Instead, we focus exclusively on the threat arising from the data exchange at the cut points, which is unique to SL and for which existing defenses are insufficient in the LLM setting considered here.

IV. Challenges and Key Insights

A. Why Perturbation-Based Defenses Fail for LLMs

Perturbation-based methods are among the most widely adopted defenses in SL for vision models. In that setting, the client often retains only the early portion of the model, while the labels are typically low-dimensional class indices that carry little information on their own. The threat is that the server trains a decoder to reconstruct the client's input from the smashed data produced by the client-side model. Perturbation-based methods counter this by injecting noise into the smashed data to reduce reconstruction quality while preserving task utility [36].

This defense strategy becomes fundamentally limited in the LLM setting. In LLM fine-tuning, the label sequence is simply the input shifted by one token, so the input and labels are nearly identical. As a result, the server can use its white-box copy of the downstream model to decode the processed smashed data and predict the labels. The difficulty is that this decoder is not an auxiliary attacker model separate from training. It is the same next-token predictor that fine-tuning is trying to make useful. Therefore, perturbing the smashed data strongly enough to make this decoder fail also

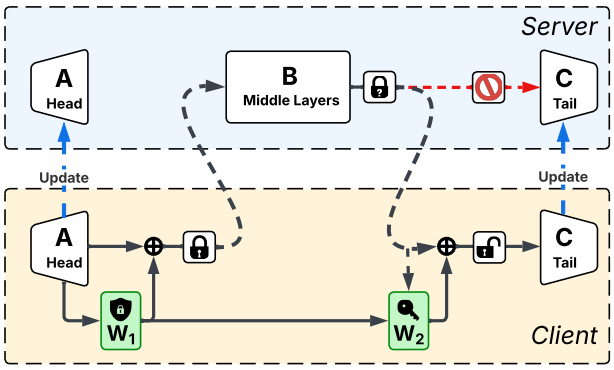


Fig. 3. The forward path of PrivPair. The client applies an obfuscation adapter W_1 to the smashed data before transmission and a recovery adapter W_2 to the returned activations. The paired W_2 is trained to recover a representation compatible with C. Without access to this recovery adapter, applying f_C directly to the obfuscated activations no longer provides an effective decoder.

makes the training path through the LLM fail. Fine-tuning further tightens this tradeoff. Given perturbed smashed data, the client's objective is still to train the model to predict the next token, which also improves the decoder available to the server once the client uploads its updated client-side parameters. The defender therefore faces a difficult tension. Stronger perturbations improve privacy but damage trainability, while weaker perturbations preserve utility but leave label reconstruction effective.

B. The PrivPair Approach

PrivPair breaks this single-representation tradeoff by making the server and the client operate on different activation spaces. Rather than adding generic perturbations that must remain usable for training, PrivPair applies a learned *obfuscation* to the smashed data before it is transmitted to the server, forcing the server-side computation to proceed in an obfuscated space that is less compatible with the server's decoder. If used alone, this obfuscation would also degrade training because C would receive activations outside its expected distribution. To preserve training effectiveness, PrivPair applies a second adapter to the activations returned by the server, mapping them toward a representation compatible with C before the backward pass proceeds on the client side. Conceptually, this forms an obfuscate-and-recover scheme. The client "*obfuscates*" the smashed data before sending it to the server, and "*recovers*" the returned activations locally, so that the server operates in the obfuscated space while the client's local computation remains aligned with the original model.

The central challenge in realizing this design is the recovery step. Due to the inherent complexity and nonlinearity of LLMs, the obfuscation applied to the smashed data propagates through the server-side model in a highly entangled manner, making it non-trivial to recover a representation compatible with C from the returned activations. The recovery adapter is therefore not assumed to be an exact inverse. Instead, PrivPair trains the two adapters jointly. The first adapter learns the forward obfuscation, and the second learns a recovery mapping that preserves training utility. The two adapters are trained simultaneously and cooperate to fulfill the obfuscation and recovery objectives.

V. Design

A. Preliminaries

We consider an FL system with N clients and one central server. Let $\mathcal{D}_i = \{(x_j, y_j)\}$ denote the private fine-tuning dataset held by client i, where x_j is an input token sequence and y_j is the corresponding label sequence. In the LLM fine-tuning setting, y_j is the shifted version of x_j, i.e., $y_j^{(t)} = x_j^{(t+1)}$.

Let the LLM consist of an embedding layer followed by L transformer blocks and a language model head. We denote the two cut points as k and m ($1 \le k < m \le L$). The model is then split into three components. A contains the embedding layer and transformer blocks 1 through k. B contains transformer blocks $k{+}1$ through m. C contains transformer blocks $m{+}1$ through L and the language model head. The full model satisfies $f = f_C \circ f_B \circ f_A$, and the loss $\mathcal{L}$ is computed at the client. The intermediate activations $s_A = f_A(x)$ and $s_B = f_B(s_A)$, and their corresponding gradients $\nabla_{s_B}\mathcal{L}$ and $\nabla_{s_A}\mathcal{L}$, are exchanged between the client and server at the two cut points.

In each training step, the forward and backward passes follow the split path $f_A \to f_B \to f_C$. The client sends s_A to the server, receives s_B, computes the loss through f_C, and exchanges the corresponding cut-point gradients with the server during backpropagation. At the end of each round, clients upload their updated client-side parameters or model updates, which the server aggregates via FedAvg before broadcasting the next global model.

The split protocol keeps B frozen and updates only A and C. Denoting the parameters of A, B, and C as θ_A, θ_B, θ_C respectively, and using learning rate η and cross-entropy loss $\ell(\cdot,\cdot)$, the adapter-free updates are given as follows.

$$s_A = f_A(x), \quad s_B = f_B(s_A)$$

$$\theta_C \leftarrow \theta_C - \eta \frac{\partial \ell\big(f_C(s_B), y\big)}{\partial \theta_C} \tag{1}$$

$$\theta_A \leftarrow \theta_A - \eta \left(\frac{\partial f_A}{\partial \theta_A}\right)^{\top} \left(\frac{\partial f_B}{\partial s_A}\right)^{\top} \frac{\partial \ell\big(f_C(s_B), y\big)}{\partial s_B} \tag{2}$$

B. Update Protocol and Adapter Design

Recall that s_A is the smashed data produced by the client and transmitted to the server, and s_B is the smashed data returned by the server to the client. PrivPair introduces two adapters, W_1 and W_2, both residing on the client.

Obfuscation adapter W_1. W_1 is implemented as a residual Multi-Layer Perceptron (MLP). Letting h_1 denote the inner feed-forward network of W_1, the obfuscation is computed as follows.

$$s'_A = W_1(s_A) = s_A + h_1(s_A) \tag{3}$$

The obfuscated smashed data s'_A is sent to the server in place of s_A. The server passes it through f_B as usual to produce $s'_B = f_B(s'_A)$, and returns s'_B to the client. Note that W_1 alone, together with its residual connection, plays the same role as perturbation-based methods, adding a bounded additive perturbation $h_1(s_A)$ to the smashed data before transmission. As with these perturbation-based methods, the intent of W_1 is not to fully hide the structure of the smashed data, but rather to add enough noise to deactivate the server's decoder.

Recovery adapter W_2. W_2 is implemented as a residual MLP that takes s'_B and the W_1 residual $\delta_A = s'_A - s_A$ as inputs. Letting h_2 denote the inner feed-forward network of W_2, the recovery is computed as follows.

$$\hat{s}_B = W_2(s'_B,\, \delta_A) = s'_B + h_2([s'_B \,\|\, \delta_A]) \tag{4}$$

where $[\cdot\|\cdot]$ denotes concatenation along the feature dimension. The client proceeds with $\hat{s}_B$ in place of s_B for the remainder of the forward and backward passes. Conditioning on δ_A rather than s'_A directly provides W_2 with a compact representation of how W_1 modified the activations. This residual assists W_2 in recovering a representation compatible with C. It is computed from s_A and s'_A on the client and is consumed only as a local input to W_2, so it is never transmitted to the server.

STE for the backward pass. To decouple adapter behavior from model training, PrivPair applies the Straight-Through Estimator (STE) [37] at both cut points. Concretely, the actual tensors passed through the network are defined as follows.

$$\tilde{s}_A = s_A + \big(s'_A - s_A\big)_{\mathrm{sg}} \tag{5}$$

$$\tilde{s}_B = s'_B + \big(\hat{s}_B - s'_B\big)_{\mathrm{sg}} \tag{6}$$

where $(\cdot)_{\mathrm{sg}}$ denotes the stop-gradient operator. In the forward pass, $\tilde{s}_A = s'_A$ and $\tilde{s}_B = \hat{s}_B$, so the obfuscated activations and the recovered activations are used as intended. In the backward pass, the stop-gradient term vanishes, so gradients flow through $\tilde{s}_A$ as if it were s_A, and through $\tilde{s}_B$ as if it were s'_B. STE ensures that $\frac{\partial \mathcal{L}}{\partial s'_B} = \frac{\partial \mathcal{L}}{\partial \tilde{s}_B}$ and $\frac{\partial \tilde{s}_A}{\partial s_A} = I$, so the gradients propagated through f_B and f_A are computed as if W_1 and W_2 were identity mappings. The STE is specifically designed to keep A trainable, which we discuss further in Section V-D. Note that perturbation-based methods, which directly add noise to the smashed data as $s'_A = s_A + \epsilon$, implicitly apply the same STE treatment in the backward pass.

With the adapters involved, the parameter updates are given as follows.

$$\tilde{s}_A = f_A(x) + \big(W_1(f_A(x)) - f_A(x)\big)_{\mathrm{sg}}$$

$$\tilde{s}_B = f_B(\tilde{s}_A) + \big(W_2(f_B(\tilde{s}_A),\, \tilde{s}_A - f_A(x)) - f_B(\tilde{s}_A)\big)_{\mathrm{sg}}$$

$$\theta_C \leftarrow \theta_C - \eta\, \frac{\partial\, \ell\big(f_C(\tilde{s}_B),\, y\big)}{\partial \theta_C} \tag{7}$$

$$\theta_A \leftarrow \theta_A - \eta \left(\frac{\partial f_A}{\partial \theta_A}\right)^{\top} \left(\frac{\partial f_B}{\partial \tilde{s}_A}\right)^{\top} \frac{\partial\, \ell\big(f_C(\tilde{s}_B),\, y\big)}{\partial f_B(\tilde{s}_A)} \tag{8}$$

θ_B is kept frozen throughout fine-tuning, and we discuss the reason in Section V-D.

C. Adapter Training

W_1 serves a dual purpose. Its primary objective is to obfuscate the smashed data into a distribution incompatible with the server's decoder to prevent label leakage, while it must simultaneously preserve enough structure to allow W_2 to enable effective training of the main model. W_2, on the other hand, is solely dedicated to enabling training. It maps the obfuscated server output back to a representation compatible with C. The training losses of W_1 and W_2 reflect these distinct roles.

Throughout this section, we follow the notation established in Section V-A, ignoring STE for now. s_A and s_B denote the smashed data produced without any adapter obfuscation. s'_A and s'_B denote the activations after W_1 has been applied but before W_2, i.e., $s'_A = W_1(s_A)$ and $s'_B = f_B(s'_A)$. $\hat{s}_B$ denotes the final output after both adapters have been applied, i.e., $\hat{s}_B = W_2(s'_B,\, \delta_A)$ where $\delta_A = s'_A - s_A$. We further denote by $z = f_C(s_B)$, $z' = f_C(s'_B)$, and $\hat{z} = f_C(\hat{s}_B)$ the logits produced by f_C under the plain, W_1-only, and full-adapter paths, respectively.

1) Loss of W_1: Let P denote the set of valid token positions, V the vocabulary size, and τ a temperature hyperparameter. For a logit tensor z, define the token-level softmax distribution $p^z_t = \mathrm{softmax}(z_t/\tau) \in \mathbb{R}^V$, and let $m_t(z, z') = \frac{1}{2}(p^z_t + p^{z'}_t)$. The token-averaged Jensen-Shannon divergence is given as follows.

$$D_{\mathrm{JSD}}(z, z') = \frac{\tau^2}{2|P|} \sum_{t \in P} \sum_{v=1}^{V} \Big(p^z_{t,v} \log \frac{p^z_{t,v}}{m_{t,v}(z, z')} + p^{z'}_{t,v} \log \frac{p^{z'}_{t,v}}{m_{t,v}(z, z')} \Big) \tag{9}$$

For the CKA term, let $\tilde{X}$ denote the column-mean-centered token matrix of X restricted to P. The linear CKA is defined as follows.

$$\mathrm{CKA}(X, Y) = \frac{\|\tilde{X}^{\top} \tilde{Y}\|_F^2}{\|\tilde{X}^{\top} \tilde{X}\|_F \cdot \|\tilde{Y}^{\top} \tilde{Y}\|_F} \tag{10}$$

The training objective of W_1 is then defined as follows.

$$\mathcal{L}_{W_1} = \lambda^{(1)}_{\mathrm{CKA}}\, \mathrm{CKA}(s'_B,\, s_B) \tag{11}$$

$$- \lambda^{(1)}_{\mathrm{JSD}}\, D_{\mathrm{JSD}}(z',\, z) \tag{12}$$

$$+ \lambda^{(1)}_{W_2}\, \mathcal{L}_{W_2} \tag{13}$$

The term (12) is the most direct privacy objective. Maximizing the JSD between z' and z reduces the server's ability to correctly decode the transmitted smashed data s'_A using $f_C \circ f_B$. The CKA term (11) provides a further and complementary signal that operates at the representation level rather than the logit level, as minimizing it pushes the geometry of s'_B away from s_B, making the intermediate activations themselves harder for the server to exploit even before reaching C. Together, the two terms reduce decodability at two different levels of the server-side model.

The term (13) couples W_1 to the restoration objective of W_2. While the privacy terms push W_1 to produce an obfuscation

that is hard to decode, (13) simultaneously encourages W_1 to produce obfuscations for which W_2 can keep the reconstruction error small. This coupling is essential. Without it, W_1 may produce transformations that are trivially obfuscated but too difficult for W_2 to map back to a trainable representation, collapsing training effectiveness. W_1 and W_2 are therefore mutually dependent and must be trained jointly. The weight $\lambda_{W_2}^{(1)}$ controls the strength of this coupling relative to the privacy terms (11) and (12). A larger $\lambda_{W_2}^{(1)}$ favors utility by biasing W_1 toward obfuscations that W_2 can more easily map back to a trainable representation, while a smaller value favors privacy by allowing W_1 to pursue stronger obfuscation at the risk of degrading W_2's recovery quality.

2) Loss of W_2: Recall that $\hat{z}$ denotes the logits produced via the full adapter path (W_1 followed by W_2), and $\hat{s}_B = W_2(s'_B, \delta_A)$ denotes the reconstructed smashed data. Following the notation above, the token-averaged KL divergence is given as follows.

$$D_{\mathrm{KL}}(z\|z') = \frac{\tau^2}{|P|}\sum_{t\in P}\sum_{v=1}^{V} p_{t,v}^{z}\log\frac{p_{t,v}^{z}}{p_{t,v}^{z'}} \tag{14}$$

The training objective of W_2 is then defined as follows.

$$\mathcal{L}_{W_2} = \lambda_{\mathrm{KL}}^{(2)}\, D_{\mathrm{KL}}\big(\hat{z}\,\|\,z\big) \tag{15}$$

$$+\,\lambda_{\mathrm{MSE}}^{(2)}\,\big\|\hat{s}_B - s_B\big\|^2 \tag{16}$$

The term (16) directly supervises W_2 to reconstruct the original smashed data s_B from the obfuscated server output s'_B. However, due to the inherent complexity and nonlinearity of the LLM, a perfect reconstruction is in general unattainable. The term (15) therefore provides a complementary signal. Even when $\hat{s}_B$ cannot exactly match s_B, minimizing $D_{\mathrm{KL}}(\hat{z}\|z)$ encourages the output distribution of f_C to match the plain-path distribution, compensating for residual reconstruction error at the distribution level. Together, the two terms help keep C trainable despite the imperfect recovery from W_1's obfuscation.

D. Trainability of the LLM Model

We now give an intuitive analysis of the trainability of A, B, and C under the adapter protocol. Formal analysis is provided in Section VI-A.

Trainability of C. Suppose W_2 is perfectly trained, i.e., $\hat{s}_B = s_B$ exactly. Then from the perspective of C, the presence of the adapters is entirely transparent. It receives the same input as in the adapter-free setting, and its training proceeds identically. Although $\hat{s}_B = s_B$ is unattainable in practice, as long as the residual error $\|\hat{s}_B - s_B\|$ is small, it can be treated as noise and the training effectiveness of C is preserved.

Trainability of B. We keep B frozen throughout training. Under the adapter protocol, B receives $s'_A = W_1(s_A)$ as input rather than s_A. Because W_1 deliberately pushes s'_A out of the distribution that B was originally trained on, any updates to θ_B would reflect the obfuscated space induced by W_1, not the original input distribution. Deploying the fine-tuned B without the adapters would expose it to out-of-distribution inputs, rendering the fine-tuning ineffective. Retaining W_1 during inference would require the client to apply it on every query, which conflicts with the standard FL deployment goal of a complete, independently usable model on the server. For fine-tuning purposes, updating A and C alone is sufficient to adapt the model to downstream tasks, as confirmed by our experiments in Section VII-D.

Trainability of A. The adapter protocol introduces a gradient deviation in A proportional to $\|s'_A - s_A\|$. If W_2 were absent, PrivPair would degenerate into a perturbation-based scheme, and trainability would follow the same condition as prior work. The introduction of W_2 relaxes this constraint when the recovery error $\|\hat{s}_B - s_B\|$ remains small, because C then receives a representation closer to the plain-path smashed data even when s'_A deviates from s_A. We provide an analysis in Section VI-C.

E. Full Training Protocol

Alignment dataset. Training W_1 and W_2 uses an arbitrary alignment dataset $\mathcal{D}_{\mathrm{align}} = \{(x_j, y_j)\}_{j=1}^{M}$ that contains no private information. This dataset is independent of the private fine-tuning dataset $\mathcal{D}_i$ and does not need to match its task, domain, or distribution. It may come from any public or synthetic text source that can be processed by the underlying model.

Adapter alignment procedure. Given $\mathcal{D}_{\mathrm{align}}$, the client runs a forward pass through the full adapter path $f_A \to W_1 \to f_B \to W_2 \to f_C$ as well as the plain path $f_A \to f_B \to f_C$. For each batch, the client computes the intermediate activations s_A, s'_A, s_B, s'_B, $\hat{s}_B$ and the corresponding logits z, z', $\hat{z}$ as defined above. The adapters are then updated using their objectives. W_1 is updated by minimizing $\mathcal{L}_{W_1}$ defined in (11) through (13), and W_2 is updated by minimizing $\mathcal{L}_{W_2}$ defined in (15) through (16). All other model parameters, including θ_A and θ_C, remain frozen throughout alignment, so the alignment procedure only ever updates W_1 and W_2. The alignment phase runs for a fixed number of steps prior to the main fine-tuning loop.

Main fine-tuning loop. After the adapter alignment phase, the system proceeds with the standard FL fine-tuning loop. At each global round r, the server selects a subset of clients $\mathcal{S}^{(r)} \subseteq \{1, \ldots, N\}$ and broadcasts the current model parameters $\theta_A^{(r)}$, $\theta_C^{(r)}$ to each selected client. Each client $i \in \mathcal{S}^{(r)}$ then performs E local steps on its private dataset $\mathcal{D}_i$. In each local step, the client runs the full adapter forward pass to obtain $\hat{z}$, computes the cross-entropy loss $\mathcal{L} = \ell(\hat{z}, y_{\mathrm{gt}})$ against the ground-truth labels y_{gt}, and updates θ_A and θ_C via the STE-based backward pass described above, with θ_B kept frozen. After E local steps, each client uploads the updated $\theta_A^{(r,i)}$ and $\theta_C^{(r,i)}$ to the server. The server then aggregates the updates via FedAvg, as follows.

$$\theta^{(r+1)} \leftarrow \sum_{i\in\mathcal{S}^{(r)}} \frac{|\mathcal{D}_i|}{\sum_{j\in\mathcal{S}^{(r)}}|\mathcal{D}_j|}\,\theta^{(r,i)} \tag{17}$$

where θ denotes θ_A and θ_C jointly. The adapters W_1 and W_2 are never uploaded and remain local to each client throughout.

Algorithm 1 PrivPair Full Training Protocol

Require: Private dataset $\mathcal{D}_i$, alignment dataset $\mathcal{D}_{\text{align}}$, initial parameters $\theta_A^{(0)}, \theta_B, \theta_C^{(0)}$, global rounds R, local steps E, alignment steps T_{align}, burst interval K

Ensure: Fine-tuned model $\theta_A^{(R)}, \theta_B, \theta_C^{(R)}$

// Initial adapter alignment
Run alignment on $\mathcal{D}_{\text{align}}$ for T_{align} steps with θ_A, θ_C frozen, and update W_1, W_2 with $\mathcal{L}_{W_1}, \mathcal{L}_{W_2}$
for each global round $r = 1, \ldots, R$ **do**
  Server broadcasts $\theta_A^{(r)}, \theta_C^{(r)}$ to selected clients $\mathcal{S}^{(r)}$
  for each client $i \in \mathcal{S}^{(r)}$ **in parallel do**
    for $e = 1, \ldots, E$ local steps **do**
      Sample batch from $\mathcal{D}_i$
      Forward: $s_A \to W_1 \to s'_A \to f_B \to s'_B \to W_2 \to \hat{s}_B \to f_C \to \hat{z}$
      Compute loss $\mathcal{L} = \ell(\hat{z}, y_{\text{gt}})$
      Backward via STE; update θ_A, θ_C (freeze θ_B)
    end for
    if $r \bmod K = 0$ **then // Burst alignment**
      Re-train W_1, W_2 on $\mathcal{D}_{\text{align}}$ for a few steps against current θ
    end if
    Upload $\theta_A^{(r,i)}, \theta_C^{(r,i)}$ to server
  end for
  Server aggregates: $\theta^{(r+1)} \leftarrow \text{FedAvg}\big(\{\theta^{(r,i)}\}_{i \in \mathcal{S}^{(r)}}\big)$
end for

In the general setting, each client independently trains and maintains its own adapter pair $(W_1^{(i)}, W_2^{(i)})$. Among mutually trusted clients, adapters may be shared when their alignment data policy permits sharing, as illustrated in Figure 1.

Because W_1 and W_2 are trained against a specific snapshot of θ_A and θ_C, they may become misaligned as the global model θ continues to accumulate updates over successive rounds. To address this, after a period of training steps, each client enters a short burst alignment phase in which W_1 and W_2 are re-trained on $\mathcal{D}_{\text{align}}$ for a few steps against the current model weights to recalibrate the adapter pair before the next training phase resumes.

Once the fine-tuning loop concludes, the server holds the complete fine-tuned model $f = f_C \circ f_B \circ f_A$, which is ready for deployment. The complete protocol is summarized in Algorithm 1.

VI. Trainability & Convergence Analysis

A. Trainability Analysis

We begin with the single-client setting, abstracting away FL and data heterogeneity. We formalize the conditions under which the adapter protocol yields an expected decrease in the plain-path loss. Let $\mathcal{L}$ denote the plain-path loss and $g_\theta^{\text{plain}} = \partial\mathcal{L}/\partial\theta$ the ideal gradient. The adapter-path gradient actually used for updates is denoted $g_\theta^{\text{adapter}}$. We assume $\mathcal{L}$ is L_F-smooth in θ, in which case a step $\theta \leftarrow \theta - \eta g_\theta^{\text{adapter}}$ with sufficiently small η decreases $\mathcal{L}$ whenever $\langle g_\theta^{\text{adapter}}, g_\theta^{\text{plain}} \rangle > 0$. A sufficient condition is

$$\|\delta g_\theta\| < \|g_\theta^{\text{plain}}\|, \quad \delta g_\theta := g_\theta^{\text{adapter}} - g_\theta^{\text{plain}}. \tag{18}$$

We make the following smoothness assumptions on f_B's Jacobian and on $\ell \circ f_C$.

Assumption 1: (i) The Jacobian J_B of f_B is L_{J_B}-Lipschitz: $\|J_B(u) - J_B(v)\|_{\text{op}} \le L_{J_B}\|u - v\|$ for all u, v, and f_B itself is L_B-Lipschitz with $\|J_B(u)\|_{\text{op}} \le L_B$. (ii) $\ell \circ f_C$ is L_C-smooth in its first argument s, and the cross derivative $\partial\nabla_{\theta_C}(\ell \circ f_C)/\partial s$ is also bounded by L_C in operator norm. (iii) The upstream gradient $g_{\text{up}} = \partial\mathcal{L}/\partial s_B$ and the parameter Jacobians $\partial f_A/\partial\theta_A$, $\partial f_C/\partial\theta_C$ have bounded operator norms uniformly along the optimization trajectory.

Such Lipschitz-smoothness assumptions are standard throughout non-convex optimization for justifying descent-based arguments, and here they are applied to f_B's Jacobian and to $\ell \circ f_C$. If they fail to hold, gradient descent itself carries no guarantee of decreasing the loss, regardless of whether PrivPair's adapters are used.

Let J_{W_1} denote the Jacobian of W_1, and let $J_{W_2}^{(1)}$, $J_{W_2}^{(2)}$ denote the partial Jacobians of W_2 with respect to its first input (s'_B) and second input (δ_A). Note that s_A enters the adapter chain through two paths: indirectly via $s'_A = W_1(s_A)$ (which feeds both f_B and δ_A), and directly via the $-s_A$ term in $\delta_A = s'_A - s_A$. Before applying STE, direct backpropagation through the adapter chain $f_A \to W_1 \to f_B \to W_2 \to f_C$ gives

$$g_{\theta_C}^{\text{adapter}} = \frac{\partial\,\ell(f_C(\hat{s}_B), y)}{\partial\theta_C}, \tag{19}$$

$$g_{\theta_A}^{\text{adapter}} = \Big(\frac{\partial f_A}{\partial\theta_A}\Big)^\top \Big[J_{W_1}(s_A)^\top J_B(s'_A)^\top J_{W_2}^{(1)}(s'_B, \delta_A)^\top + \big(J_{W_1}(s_A)^\top - I\big) J_{W_2}^{(2)}(s'_B, \delta_A)^\top\Big] \frac{\partial\mathcal{L}}{\partial\hat{s}_B}. \tag{20}$$

The corresponding plain-path gradients are

$$g_{\theta_C}^{\text{plain}} = \partial\,\ell(f_C(s_B), y)/\partial\theta_C \text{ and}$$

$$g_{\theta_A}^{\text{plain}} = (\partial f_A/\partial\theta_A)^\top J_B(s_A)^\top \partial\mathcal{L}/\partial s_B.$$

Comparing (20) against the plain-path expression, the deviation δg_{θ_A} contains four sources, namely (i) J_{W_1} deviating from identity, (ii) $J_{W_2}^{(1)}$ deviating from identity, (iii) $J_{W_2}^{(2)}$ deviating from zero, and (iv) the mismatch between $\partial\mathcal{L}/\partial\hat{s}_B$, $J_B(s'_A)$ and $\partial\mathcal{L}/\partial s_B$, $J_B(s_A)$. The coupling term $(J_{W_1}^\top - I)\, J_{W_2}^{(2)\top}$ in (20) vanishes whenever either (i) or (iii) is suppressed.

Under STE, sources (i)–(iii) vanish. The stop-gradient construction of Section V enforces

$$J_{W_1}(s_A) = I, \quad J_{W_2}^{(1)}(s'_B, \delta_A) = I, \quad J_{W_2}^{(2)}(s'_B, \delta_A) = 0 \tag{21}$$

in the backward pass, independent of the actual functional form of W_1 and W_2. When substituting into (20), only source

(iv) remains, and it decomposes into an upstream gradient error and a Jacobian evaluation error:

$$\delta g_{\theta_A} = \Big(\frac{\partial f_A}{\partial \theta_A}\Big)^\top \Big[J_B(s'_A)^\top \Big(\tfrac{\partial \mathcal{L}}{\partial \hat{s}_B} - \tfrac{\partial \mathcal{L}}{\partial s_B}\Big) + \big(J_B(s'_A) - J_B(s_A)\big)^\top \tfrac{\partial \mathcal{L}}{\partial s_B}\Big]. \tag{22}$$

Using Assumption 1 on (22), and letting $g_{\text{up}} = \partial \mathcal{L}/\partial s_B$, we obtain

$$\|\delta g_{\theta_A}\| \le \Big\|\tfrac{\partial f_A}{\partial \theta_A}\Big\|_{\text{op}} \Big[\|J_B(s'_A)\|_{\text{op}} \cdot L_C \|\hat{s}_B - s_B\| + \|g_{\text{up}}\| \cdot L_{J_B} \|s'_A - s_A\| \Big]. \tag{23}$$

An analogous bound for θ_C gives $\|\delta g_{\theta_C}\| \le L_C \|\hat{s}_B - s_B\|$.

Combining (18) and (23) yields the sufficient condition for θ_A:

$$\|J_B(s'_A)\|_{\text{op}} \cdot L_C \|\hat{s}_B - s_B\| + \|g_{\text{up}}\| \cdot L_{J_B} \|s'_A - s_A\| < \frac{\|g^{\text{plain}}_{\theta_A}\|}{\|\partial f_A / \partial \theta_A\|_{\text{op}}}, \tag{24}$$

and analogously $L_C \|\hat{s}_B - s_B\| < \|g^{\text{plain}}_{\theta_C}\|$ for θ_C. The first error term in (24) is directly minimized by the W_2 loss (16), while the second is kept finite by the residual structure of W_1 together with the coupling term (13) described in Section V-B.

B. Convergence Analysis

We now extend to the full FL setting with N clients and non-i.i.d. data. Our convergence analysis builds on the standard FedAvg convergence framework of [38], adopting Assumptions 2 through 6 below to characterize the underlying FL optimization problem, and introduces an additional smashed data deviation constraint, Assumption 7. If any of Assumptions 2 through 6 fails to hold, standard FL itself is not guaranteed to converge, regardless of whether PrivPair is used. We first analyze the case where θ_A is frozen and only θ_C is trained. We extend to joint (θ_A, θ_C) training at the end of this section. Throughout, θ refers to θ_C unless stated otherwise, and θ_B is frozen throughout (see Section V). Let $F_i(\theta)$ denote the local objective at client i under the plain (adapter-free) path.

Assumption 2: Local objective functions $F_1, \ldots, F_N$ are all L-smooth: for all v and w, $F_i(v) \le F_i(w) + (v-w)^T \nabla F_i(w) + \frac{L}{2}\|v-w\|_2^2$.

Assumption 3: $F_1, \ldots, F_N$ are all μ-strongly convex: for all v and w, $F_i(v) \ge F_i(w) + (v-w)^T \nabla F_i(w) + \frac{\mu}{2}\|v-w\|_2^2$.

We adopt Assumption 3, together with Assumptions 2, 4–6, from the FedAvg analysis of [38] to characterize the underlying FL optimization problem in a setting where a convergence guarantee is analytically tractable, following standard practice in the FL convergence literature. Like this and other such analyses, it does not literally hold for the non-convex loss landscape of LLM fine-tuning. Within this same idealized setting, our point is that PrivPair introduces no convergence obstacle beyond what the underlying FedAvg baseline already requires.

Assumption 4: Let ξ_i be sampled from the i-th device's local data uniformly at random. The variance of stochastic gradients in each device is bounded: $\mathbb{E}\|\nabla F_i(\xi_i;\theta_i^t) - \nabla F_i(\theta_i^t)\|^2 \le \sigma_i^2$ for $i = 1, \ldots, N$.

Assumption 5: The expected squared norm of stochastic gradients is uniformly bounded, i.e., $\mathbb{E}\|\nabla F_i(\xi_i;\theta_i^t)\|^2 \le G^2$ for all $i = 1, \ldots, N$ and $t = 1, \ldots, T-1$.

Assumption 6: Assume S_t contains a subset of K indices uniformly randomly from [N] without replacement. Assume the data is balanced in the sense that $p_1 = \cdots = p_N = \frac{1}{N}$. The aggregation step of FedAvg performs $\theta_t \leftarrow \frac{N}{K}\sum_{i \in S_t} p_i \theta_i^t$.

Assumption 7: The expected squared norm of the difference between the plain-path smashed data s_B and the recovered smashed data $\hat{s}_B$ is bounded. Specifically, $\mathbb{E}\|s_B - \hat{s}_B\|^2 \le H$ for all $i = 1, \ldots, N$ and $t = 1, \ldots, T-1$.

By Assumption 1 and the bound (23), there exists a constant $L_s > 0$ depending only on the quantities in Assumption 1 such that, in the θ_C-only setting,

$$\mathbb{E}\big\|\tilde{\nabla} F_i(\xi_i;\theta) - \nabla F_i(\xi_i;\theta)\big\|^2 \le L_s^2 \,\mathbb{E}\big\|s_B - \hat{s}_B\big\|^2, \tag{25}$$

where $\tilde{\nabla} F_i$ denotes the adapter-path gradient and ∇F_i denotes the plain-path gradient at the same parameters. For θ_C, $L_s = L_C$ follows directly from the cross-derivative bound in Assumption 1.

Now we derive a variance bound that accounts for the adapter-path gradient in place of Assumption 4. Let $\delta_i^t = \tilde{\nabla} F_i(\xi_i;\theta_i^t)$. By the elementary inequality $\|a+b\|^2 \le 2\|a\|^2 + 2\|b\|^2$ together with (25) and Assumptions 4, 7,

$$\begin{aligned} &\mathbb{E}\,\big\|\delta_i^t - \nabla F_i(\theta_i^t)\big\|^2 \\ &\le 2\,\mathbb{E}\,\big\|\delta_i^t - \nabla F_i(\xi_i;\theta_i^t)\big\|^2 + 2\,\mathbb{E}\,\big\|\nabla F_i(\xi_i;\theta_i^t) - \nabla F_i(\theta_i^t)\big\|^2 \\ &\le 2L_s^2\,\mathbb{E}\,\|s_B - \hat{s}_B\|^2 + 2\sigma_i^2 \\ &\le 2(\sigma_i^2 + L_s^2 H). \end{aligned} \tag{26}$$

Note that this bound subsumes both the variance and the bias of δ_i^t relative to $\nabla F_i(\theta_i^t)$, since $\mathbb{E}\|\delta - \nabla F\|^2 = \|\mathbb{E}\delta - \nabla F\|^2 + \mathrm{Var}(\delta)$. The same argument yields a corresponding squared-norm bound in place of Assumption 5, namely $\mathbb{E}\,\|\delta_i^t\|^2 \le 2(G^2 + L_s^2 H)$ for all $i = 1, \ldots, N$ and $t = 1, \ldots, T-1$.

We define F^* and F_i^* as the minimum values of F and F_i and $\Gamma = F^* - \sum_{i=1}^N p_i F_i^*$. We assume each device has E local updates. The bound above shows that the adapter-path update can be viewed as a stochastic update whose deviation from the plain-path gradient is controlled by the recovery error H. Applying the FedAvg convergence argument of [38] with the gradient-deviation bound $2(\sigma_i^2 + L_s^2 H)$ and the squared-norm bound $2(G^2 + L_s^2 H)$ gives the following bound on non-i.i.d. data.

Theorem 6.1:

Let Assumption 1 and Assumptions 2–7 hold, and let $L, \mu, \sigma_i, G, K, H, L_s$ be defined therein. Choose $\kappa = \frac{L}{\mu}, \gamma = \max\{8\kappa, E\}$ and the learning rate $\eta_t = \frac{2}{\mu(\gamma+t)}$.

Then

$$\mathbb{E}[F(\theta_T)]-F^* \le \frac{\kappa}{\gamma+T-1}\left(\frac{2(P+Q)}{\mu}+\frac{\mu\gamma}{2}\mathbb{E}\left\|\theta_1-\theta^*\right\|^2\right) \tag{27}$$

where

$$P = 2\sum_{i=1}^{N} p_i^2(\sigma_i^2+L_s^2H)+6L\Gamma+16(E-1)^2(G^2+L_s^2H), \tag{28}$$

$$Q = \frac{N-K}{N-1}\frac{8}{K}E^2(G^2+L_s^2H). \tag{29}$$

Extension to joint (θ_A, θ_C) training. When θ_A is also trained, the adapter-path gradient deviation for θ_A contains an additional term proportional to $\|s'_A - s_A\|$ (cf. (23)), which is not captured by Assumption 7. To handle this, we introduce one further assumption.

Assumption 8: The expected squared norm of the difference between the plain-path smashed data s_A and the obfuscated smashed data s'_A is bounded. Specifically, $\mathbb{E}\left\|s_A - s'_A\right\|^2 \le H_A$ for all $i = 1, \dots, N$ and $t = 1, \dots, T-1$.

Under Assumption 1 and Assumptions 2–8, applying the bound (23) together with $(a+b)^2 \le 2a^2+2b^2$ shows that there exists $L_s > 0$ such that

$$\mathbb{E}\left\|\tilde{\nabla}F_i(\xi_i;\theta)-\nabla F_i(\xi_i;\theta)\right\|^2 \le L_s^2\,(H+H_A). \tag{30}$$

The derivation then proceeds as before, with the gradient variance bound replaced by $2\big(\sigma_i^2+L_s^2(H+H_A)\big)$ and the squared gradient norm bound replaced by $2\big(G^2+L_s^2(H+H_A)\big)$. The theorem above then holds with P and Q replaced by

$$P = 2\sum_{i=1}^{N} p_i^2\big(\sigma_i^2+L_s^2(H+H_A)\big)+6L\Gamma + 16(E-1)^2\big(G^2+L_s^2(H+H_A)\big), \tag{31}$$

$$Q = \frac{N-K}{N-1}\frac{8}{K}E^2\big(G^2+L_s^2(H+H_A)\big). \tag{32}$$

C. Comparison with Perturbation-Based Defenses

In a perturbation-based scheme, $s'_A = s_A + \varepsilon$, and the client feeds $s'_B = f_B(s'_A)$ directly to C. In this case, the upstream gradient error in (23) is $L_C\|s'_B - s_B\|$, and the convergence bound contains $H_{\text{perturb}} = \mathbb{E}\|s'_B - s_B\|^2$ in place of H. By Assumption 1, f_B is L_B-Lipschitz, so

$$H_{\text{perturb}} = \mathbb{E}\|s'_B - s_B\|^2 \le L_B^2\,H_A, \tag{33}$$

That is, H_{perturb} is *implicitly* controlled by H_A. The two are coupled through L_B, and there is only one degree of freedom. Consequently, in any perturbation-based scheme the convergence bound contains $L_s^2(H_{\text{perturb}}+H_A) \le L_s^2(L_B^2+1)H_A$, and both error terms in (24) also grow together as the perturbation magnitude $\|\varepsilon\|$ increases. Strengthening the privacy protection through a larger ε thus directly worsens the convergence-rate constants P, Q and tightens the descent margin in (24).

In PrivPair, $H = \mathbb{E}\|\hat{s}_B - s_B\|^2$ is *explicitly* controlled by the W_2 loss (16), while $H_A = \mathbb{E}\|s'_A - s_A\|^2$ is a separate quantity shaped by the residual structure of W_1 together with the coupling term (13). Thus the training quality depends on the achieved pair (H, H_A) rather than on H_A alone. When the recovery adapter attains a small value of H, a larger deviation at the first cut point can be tolerated without proportionally increasing the error seen by C. The advantage of PrivPair is therefore captured by Assumptions 7 and 8. PrivPair decouples H_A from H, unlike the perturbation-based case where the two are tied together through L_B, so privacy can be strengthened as long as W_2 keeps the corresponding reconstruction error H small.

VII. Evaluation

A. Experimental Setup

Models. We evaluate four instruction-tuned LLMs. The model suite includes Llama-3.2-3B-Instruct and Llama-3.1-8B-Instruct [39], together with Ministral-3-8B-Instruct and Ministral-3-14B-Instruct [40]. These models span two model families and parameter counts from 3B to 14B. We evaluate downstream task performance for every model and dataset combination.

Datasets. We evaluate three downstream datasets. Banking77 [41] contains online banking requests labeled with 77 fine-grained customer intents. CLINC150 [42] covers 150 intent classes across multiple service domains. MentalChat16K [43] contains mental health dialogues for counseling-style response generation. Together, these three datasets span single-domain classification, multi-domain classification, and open-ended dialogue generation, letting us test the adapter protocol across different task types and domains. The client's input text is sensitive in all three cases, whether it is a banking request, a service inquiry, or a personal dialogue, and we evaluate both defense capability and training effectiveness on all three datasets. All training, evaluation, and alignment subsets used within each downstream task are non-overlapping. In the FL experiments, the private training data are partitioned into non-overlapping local subsets across clients. For the alignment dataset $\mathcal{D}_{\text{align}}$ used to train W_1 and W_2 (Section V-C), we use OASST1 [44], a crowd-sourced corpus of multi-turn assistant conversations. OASST1 is unrelated to Banking77, CLINC150, and MentalChat16K in both task and domain, which lets us verify that the adapters can be trained without any access to data resembling the private fine-tuning task.

Baselines. We compare PrivPair against three baseline defenses, all reproduced under the same SL pipeline.

Embedding ε-Privacy [24] applies noise at the embedding layer. A unit norm direction $\mathbf{v} = \mathcal{N}(0, I_d)/\|\mathcal{N}(0, I_d)\|$ and a magnitude $r \sim \text{Gamma}(d, \varepsilon d)$ are sampled, where the second parameter is the rate. The noisy embedding $\tilde{e} = e + r\mathbf{v}$ is then mapped to the vocabulary token with the largest embedding dot product, replacing the noisy embedding with the corresponding token embedding.

TABLE I
DEFENSE CAPABILITY COMPARISON. EACH CELL REPORTS THE MEAN ROUGE-1 OF RECONSTRUCTED PRIVATE INPUTS UNDER THE CORRESPONDING ATTACK, MEASURED AT THE FINAL CHECKPOINT. HIGHER ROUGE-1 INDICATES STRONGER LEAKAGE, AND LOWER IS BETTER FOR THE DEFENSE. DF DENOTES DIRECT FORWARD. BOLDFACE MARKS THE BEST DEFENSE IN EACH COLUMN.

		Banking77				**CLINC150**				**MentalChat16K**			
Model	**Method**	**DF**	**TAG**	**LAMP**	**BiSR**	**DF**	**TAG**	**LAMP**	**BiSR**	**DF**	**TAG**	**LAMP**	**BiSR**
Llama-3.2-3B	No Defense	0.1223	0.5336	0.7728	0.9991	0.0644	0.8001	0.8103	1.0000	0.6488	0.9900	0.9927	0.9916
	Embedding ε-Privacy	0.0572	0.6447	0.7119	0.8181	0.0266	0.6506	0.5524	0.8140	0.1895	0.9971	0.9969	0.9876
	Smashed-Data DP	0.1095	0.5695	0.7749	0.9991	0.0532	0.7305	0.7973	0.9994	0.3337	0.9970	0.9960	0.9956
	NoPeek	0.1183	0.5547	0.8573	0.9428	0.0740	0.7602	0.8595	1.0000	0.5453	0.9954	0.9938	0.9998
	PrivPair (ours)	**0.0000**	**0.0000**	**0.0000**	**0.0000**	**0.0000**	**0.0000**	**0.0000**	**0.0680**	**0.0000**	**0.0000**	**0.0000**	**0.1126**
Llama-3.1-8B	No Defense	0.1239	0.9814	0.9790	0.9670	0.0809	0.9821	0.9820	0.9794	0.6716	0.9885	0.9857	0.9900
	Embedding ε-Privacy	0.0368	0.9682	0.9689	0.9576	0.0072	0.9699	0.5827	0.9694	0.2579	0.9973	0.9957	0.9903
	Smashed-Data DP	0.0832	0.9002	0.9283	0.9680	0.0431	0.9777	0.9852	0.9794	0.2805	0.9961	0.9953	0.9894
	NoPeek	0.0411	0.9840	0.9843	0.9735	0.0017	0.8612	0.1069	0.7928	0.0974	0.9829	0.3686	0.6169
	PrivPair (ours)	**0.0000**	**0.0047**	**0.0036**	**0.0801**	**0.0000**	**0.0000**	**0.0000**	**0.0425**	**0.0006**	**0.0507**	**0.0124**	**0.2658**
Ministral-3-8B	No Defense	0.1137	0.8672	0.8703	0.9640	0.0508	0.9018	0.9053	0.9810	0.7071	0.9967	0.9971	0.9479
	Embedding ε-Privacy	0.0639	0.8542	0.6279	0.9216	0.0255	0.7387	0.3046	0.9363	0.4904	0.9938	0.9915	0.9326
	Smashed-Data DP	0.0878	0.8792	0.8925	0.9665	0.0169	0.8474	0.8374	0.9758	0.4712	0.9873	0.9894	0.9491
	NoPeek	0.1163	0.8787	0.8798	0.9582	0.0538	0.8916	0.4406	0.9831	0.0640	0.2868	0.1262	**0.1365**
	PrivPair (ours)	**0.0016**	**0.0021**	**0.0000**	**0.0000**	**0.0003**	**0.0013**	**0.0000**	**0.0000**	**0.0000**	**0.0016**	**0.0000**	0.1807
Ministral-3-14B	No Defense	0.1270	0.8843	0.8903	1.0000	0.0786	0.9164	0.8386	0.9993	0.6995	0.9974	0.9977	0.9963
	Embedding ε-Privacy	0.0316	0.8579	0.7107	0.9494	0.0249	0.8096	0.8000	0.9149	0.2228	0.9953	0.9961	0.9978
	Smashed-Data DP	0.0478	0.8938	0.8996	1.0000	0.0469	0.9435	0.5613	1.0000	0.4142	0.9884	0.9854	0.9913
	NoPeek	0.0994	0.8877	0.8903	0.8940	0.0653	0.9225	0.7919	0.9808	0.1939	0.9939	0.8225	0.9941
	PrivPair (ours)	**0.0011**	**0.0041**	**0.0000**	**0.1206**	**0.0000**	**0.0025**	**0.0027**	**0.1306**	**0.0014**	**0.0076**	**0.0000**	**0.2730**

Smashed-Data DP [26] applies Gaussian noise directly to the smashed data, as follows.

$$s'_A = s_A + \sigma \cdot \mathcal{N}(0, I) \tag{34}$$

where σ controls the noise magnitude.

NoPeek [29] adds a distance correlation regularization term to the training objective, as follows.

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \cdot \text{dCor}(e,\, s_A) \tag{35}$$

where e denotes the input embedding, s_A the smashed data, and λ the regularization weight.

Table VI in Appendix A lists the hyperparameter values used for each baseline. Each of these hyperparameters governs the same privacy-utility trade-off as $\lambda^{(1)}_{W_2}$ does in our method, so we tuned them carefully. Appendix B describes the tuning procedure and reports results across the tuning range. We additionally include *No Defense* as a privacy and utility reference. All split methods in this evaluation, including No Defense, the three baselines, and PrivPair, freeze B and update only A and C. *Full Model* fine-tunes the entire network without a split and is reported as a utility reference.

Defense capability setup. To evaluate defense against DRA, we follow prior work [23] and adopt the worst-case single-client setting, in which the server has full access to all smashed data and the model parameters from a client with no interference from other participants. This access excludes the client-side adapters W_1 and W_2, which remain local under our threat model. The setup maximizes the adversary's advantage within our stated scope and provides a strong attack surface for evaluating PrivPair's privacy protection.

Training effectiveness setup. In addition to evaluating training effectiveness in the single-client setting, we also measure it under FL conditions. For this, we use 64 clients, each holding its own local dataset partition, randomly assigned into 8 groups that each share an adapter pair (W_1, W_2), emulating institutions in which multiple client devices operate under a common adapter policy. Each adapter pair remains local to its group throughout training and is never shared with the server or with clients outside the group. In each of 256 global rounds, 8 clients are selected to each perform 4 local steps, after which the server aggregates the shared A and C parameters across the selected clients via FedAvg. The alignment data used by each group is also non-overlapping with that of every other group.

Hyperparameters and hardware. Table IV in Appendix A summarizes the hyperparameters used across all experiments, including the learning rate settings that vary by dataset and the per-model adapter coupling weight. The main experiments are conducted on an NVIDIA RTX PRO 6000 with 96 GB GPU memory. Client-side overhead, including adapter inference and memory consumption during fine-tuning, is measured on an NVIDIA Jetson Orin Nano with 8 GB unified memory to reflect edge device constraints, as shown in Figure 4.

B. Evaluation Methodology

We evaluate PrivPair from three perspectives.

TABLE II
TRAINING EFFECTIVENESS COMPARISON. BANKING77 AND CLINC150 REPORT CLASSIFICATION ACCURACY. MENTALCHAT16K REPORTS ROUGE-1. FULL MODEL FINE-TUNES THE ENTIRE NETWORK WITHOUT A SPLIT. BOLDFACE MARKS THE BEST DEFENSE IN EACH COLUMN.

Method	Banking77	CLINC150	MentalChat
Llama-3.2-3B			
Base	0.3945	0.4453	0.3676
No Defense	0.6436	0.7471	0.3916
Full Model	0.6445	0.7480	0.3887
Emb ε-Priv.	0.5117	0.5557	0.3843
Smashed DP	0.4883	0.5596	0.3857
NoPeek	0.4482	0.6777	0.3246
PrivPair (ours)	0.6221	0.7168	0.3878
PrivPair (FL)	**0.6406**	**0.7236**	**0.3923**
Llama-3.1-8B			
Base	0.5430	0.6123	0.3626
No Defense	0.7090	0.8359	0.3932
Full Model	0.7109	0.8340	0.4007
Emb ε-Priv.	0.6240	0.6338	0.3837
Smashed DP	0.5254	0.6016	**0.3912**
NoPeek	0.0000	0.0000	0.0300
PrivPair (ours)	**0.6641**	0.7910	0.3911
PrivPair (FL)	0.6543	**0.8018**	0.3826
Ministral-3-8B			
Base	0.6650	0.7109	0.2670
No Defense	0.7793	0.8936	0.4038
Full Model	0.8037	0.8955	0.4135
Emb ε-Priv.	0.7549	0.8623	0.3949
Smashed DP	0.6699	0.7686	0.3787
NoPeek	0.6699	0.8467	0.0017
PrivPair (ours)	0.7588	0.8779	**0.4062**
PrivPair (FL)	**0.7715**	**0.8818**	0.3977
Ministral-3-14B			
Base	0.6758	0.7568	0.2584
No Defense	0.7686	0.8672	0.3970
Full Model	0.7842	0.8955	0.4080
Emb ε-Priv.	0.7246	0.8262	0.3270
Smashed DP	0.7070	0.8477	0.3755
NoPeek	0.7266	0.8281	0.1210
PrivPair (ours)	**0.7510**	0.8760	**0.4003**
PrivPair (FL)	**0.7510**	**0.8789**	0.3800

Defense capability. We assess the privacy protection provided by PrivPair by measuring how well an adversary can reconstruct the client's private training data. We evaluate against four attacks, *Direct Forward*, *TAG*, *LAMP*, and *BiSR*. Under Direct Forward, the server directly applies $f_C \circ f_B$ to the received smashed data to decode the output. This is the most natural and zero-cost attack available to the server. TAG, LAMP, and BiSR correspond to stronger adversaries that rely on optimization or search and were originally designed for gradient- and embedding-based reconstruction attacks against LLMs. BiSR's inversion decoder is trained on A's output s_A on an auxiliary dataset. Under our defense, we do not retrain this decoder on W_1's output s'_A. Doing so would require the adversary to produce paired examples of known inputs and the corresponding s'_A on an auxiliary dataset, but W_1 never leaves the client. For a fair comparison, we also apply the same A-trained inversion decoder when evaluating BiSR against the other baselines. We report the mean ROUGE-1 of the reconstructed private inputs as the primary reconstruction quality metric.

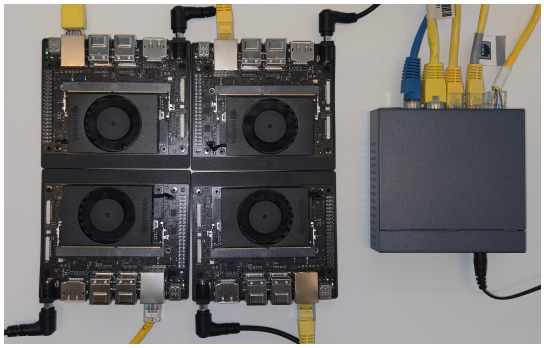


Fig. 4. Physical testbed used to evaluate PrivPair. With PrivPair, edge devices such as the NVIDIA Jetson Orin Nano can serve as clients in FL-based LLM fine-tuning.

All four attacks in Table I are measured at the final checkpoint. Direct Forward is also tracked throughout training. TAG, LAMP, and BiSR are computationally intensive and are evaluated at the final checkpoint only.

Training effectiveness. We measure whether the adapter protocol preserves the utility of fine-tuning. Since Banking77 and CLINC150 are intent classification tasks, we report classification accuracy on their evaluation sets. Since MentalChat16K is a free-form generation task, we report ROUGE-1 on its evaluation set. We compare the pre-trained base model, plain SL without privacy protection, full-model fine-tuning, the baselines, PrivPair in the single-client setting, and PrivPair in the FL setting under these metrics.

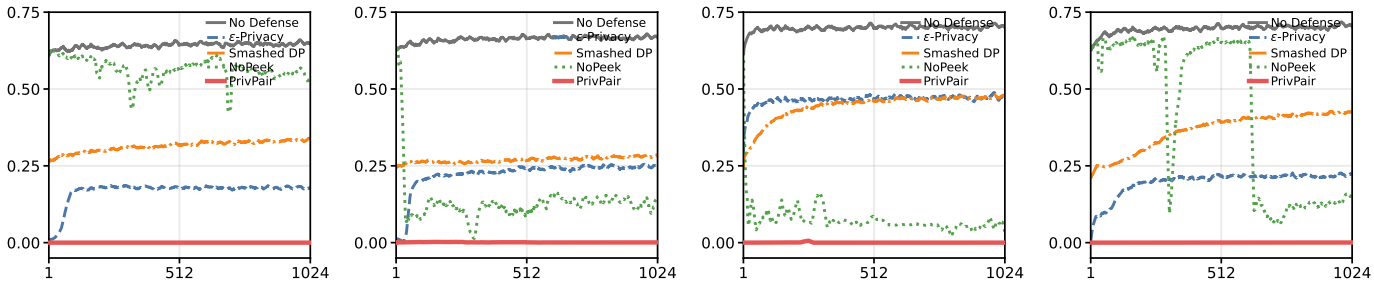
Overhead. We measure two overhead metrics. The first is the per-step client-side computation time and memory footprint introduced by running W_1 and W_2 during fine-tuning, relative to the plain SL baseline. The second is the end-to-end wall-clock training time.

C. Defense Capability

Table I reports the ROUGE-1 of reconstructed private inputs under each attack. Without any defense, Direct Forward attains high reconstruction quality on MentalChat16K, showing that the server's available decoder poses an immediate threat on long-form inputs. Direct Forward scores are lower on Banking77 and CLINC150 because the private texts are short, but TAG, LAMP, and BiSR remain high on those datasets as well, so standard SL still exhibits substantial reconstruction leakage under stronger attacks.

The baselines can reduce Direct Forward scores, but TAG, LAMP, and BiSR remain high in most settings. Some NoPeek entries show lower attack scores together with collapsed training utility in Table II.

PrivPair substantially reduces reconstruction scores. Direct Forward, TAG, and LAMP stay near zero. BiSR remains the most difficult attack for PrivPair to suppress. Across the evaluated settings, PrivPair substantially outperforms the baselines in the vast majority of attack columns.



(a) Llama-3.2-3B (b) Llama-3.1-8B (c) Ministral-3-8B (d) Ministral-3-14B

Fig. 5. Sixteen-step moving mean Direct Forward ROUGE-1 on MentalChat16K throughout training under different defense conditions.

D. Training Effectiveness

Table II reports classification accuracy on Banking77 and CLINC150, and ROUGE-1 on MentalChat16K. Among the baselines, training effectiveness varies across methods, models, and datasets. Embedding ε-Privacy and Smashed-Data DP often fall below the No Defense split baseline, especially on the classification tasks. NoPeek collapses on several MentalChat16K settings.

PrivPair achieves training effectiveness close to the no-defense references across models and datasets. The results for PrivPair (FL) are comparable to the single-client setting. This indicates that the per-client adapter design can scale to federated conditions in our setup without clear degradation from client-specific adapters.

Taken together with Table I, on the vast majority of model-dataset combinations PrivPair simultaneously provides stronger privacy protection than every baseline and matches or exceeds their training effectiveness, rather than trading one for the other. In contrast, the baselines in Table II that come closest to PrivPair's privacy protection, such as NoPeek on several MentalChat16K settings, do so only by collapsing training utility.

E. Direct Forward Attack During Training

Figure 5 shows how the Direct Forward ROUGE-1 evolves over the course of fine-tuning on MentalChat16K for every model in the model suite. For No Defense, Embedding ε-Privacy, and Smashed-Data DP, the reconstruction quality rises as training progresses. This increase follows from the fine-tuning objective, since training the LLM to predict the next token given the smashed data also improves the decoder available to the server for the client's input. Defenses that rely solely on perturbing the smashed data are limited by this effect, because the perturbation must remain bounded to preserve training utility while the server's available decoder adapts over time. PrivPair stays near zero on every model throughout training, showing that the obfuscate-and-recover scheme remains stable as fine-tuning proceeds.

F. Overhead

1) Training Overhead: The adapter pair W_1 and W_2 introduces only a small additional memory footprint relative to the base client-side model. On the NVIDIA Jetson Orin Nano, the A and C components of Llama-3.2-3B occupy 2,655 MB of GPU memory, while the two adapters together require only 144 MB, amounting to approximately 5.4% of the base memory cost. Furthermore, since the adapters do not participate in the main computation graph under the STE protocol, they do not incur additional activation memory during backpropagation. We measure the per-step client-side computation time on the NVIDIA Jetson Orin Nano, evaluating only the client-side components A and C. Processing 16 samples (sequence length 64, batch size 1) takes 3.70 seconds without adapters and 3.74 seconds with adapters. The small latency increase indicates that running W_1 and W_2 on a resource-constrained edge device introduces limited computational overhead during fine-tuning. During the main fine-tuning steps, the smashed data transmitted to the server remains a single tensor of the same shape as in the plain SL baseline, and the model update upload cost is unchanged, so running W_1 and W_2 on the client introduces no additional communication overhead beyond the alignment cost.

2) End-to-End Training Time: Table III reports the measured wall-clock training time on MentalChat16K. Excluding the one-time initial alignment cost, PrivPair's main training loop, which already includes the recurring burst alignment steps, is 5% to 10% slower than No Defense, a range comparable to or smaller than the overhead of NoPeek and Embedding ε-Privacy. Smashed-Data DP adds only a fixed noise term to the smashed data and, as expected, shows close to zero overhead. Including the one-time initial alignment cost, PrivPair's total training time is around 40% higher than No Defense, and this share shrinks as the fine-tuning loop grows longer, since the initial cost does not scale with the number of training steps.

VIII. Conclusion

We presented PrivPair, a privacy-preserving framework for LLM fine-tuning based on federated split learning that addresses the tension between smashed-data privacy and trainability. We showed that the autoregressive nature of LLMs makes perturbation-based defenses insufficient in this setting, since fine-tuning can improve the decoder available to the server while the perturbation must remain bounded to pre-

TABLE III

MEASURED WALL-CLOCK TRAINING TIME, ALL ON A SINGLE GPU. PERCENTAGES REPORT THE OVERHEAD RELATIVE TO NO DEFENSE. PRIVPAIR (TRAIN) IS THE MAIN FINE-TUNING LOOP, INCLUDING RECURRING BURST ALIGNMENT. PRIVPAIR (INIT ALIGN) IS THE ONE-TIME INITIAL ALIGNMENT COST. PRIVPAIR (TOTAL) SUMS THE TWO.

Model	No Defense	Emb ε-Priv.	Smashed DP	NoPeek	PrivPair (train)	PrivPair (init align)	PrivPair (total)
Llama-3.2-3B	13.84 min	16.90 min (+22.1%)	14.10 min (+1.9%)	15.57 min (+12.5%)	14.50 min (+4.8%)	4.80 min	19.30 min (+39.5%)
Llama-3.1-8B	27.61 min	31.43 min (+13.8%)	27.18 min (-1.5%)	29.55 min (+7.0%)	29.20 min (+5.8%)	8.95 min	38.16 min (+38.2%)
Ministral-3-8B	29.12 min	33.08 min (+13.6%)	28.70 min (-1.4%)	31.00 min (+6.5%)	30.72 min (+5.5%)	9.42 min	40.14 min (+37.8%)
Ministral-3-14B	50.26 min	61.67 min (+22.7%)	51.00 min (+1.5%)	57.07 min (+13.6%)	53.78 min (+7.0%)	16.05 min	69.83 min (+38.9%)

serve utility. Our core contribution is a learned obfuscate-and-recover protocol implemented by two lightweight client-side adapters. W_1 makes the transmitted activations less compatible with the server's decoder, while W_2 maps the returned activations toward a representation compatible with the client-side model. The adapters remain local to the client because they are never uploaded. An STE-based backward pass preserves effective updates to the deployable model. Our analysis characterizes the trainability conditions of the adapter protocol and shows how training quality depends on the achieved obfuscation and recovery errors. Our defense evaluation across four LLMs and three downstream datasets demonstrates that PrivPair substantially reduces reconstruction quality under the evaluated attacks while maintaining utility close to the no-defense references.

REFERENCES

[1] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," in *International Conference on Learning Representations*, 2022.

[2] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 27 730–27 744.

[3] P. Vepakomma, O. Gupta, T. Swedish, and R. Raskar, "Split learning for health: Distributed deep learning without sharing raw patient data," 2018.

[4] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*. Pmlr, 2017, pp. 1273–1282.

[5] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. Nitin Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, R. G. L. D'Oliveira, H. Eichner, S. El Rouayheb, D. Evans, J. Gardner, Z. Garrett, A. Gascón, B. Ghazi, P. B. Gibbons, M. Gruteser, Z. Harchaoui, C. He, L. He, Z. Huo, B. Hutchinson, J. Hsu, M. Jaggi, T. Javidi, G. Joshi, M. Khodak, J. Konecný, A. Korolova, F. Koushanfar, S. Koyejo, T. Lepoint, Y. Liu, P. Mittal, M. Mohri, R. Nock, A. Özgür, R. Pagh, H. Qi, D. Ramage, R. Raskar, M. Raykova, D. Song, W. Song, S. U. Stich, Z. Sun, A. T. Suresh, F. Tramèr, P. Vepakomma, J. Wang, L. Xiong, Z. Xu, Q. Yang, F. X. Yu, H. Yu, and S. Zhao, "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, no. 1–2, p. 1–210, Jun. 2021.

[6] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, 2020.

[7] M. Aledhari, R. Razzak, R. M. Parizi, and F. Saeed, "Federated learning: A survey on enabling technologies, protocols, and applications," *IEEE Access*, vol. 8, pp. 140 699–140 725, 2020.

[8] Q. Duan, J. Huang, S. Hu, R. Deng, Z. Lu, and S. Yu, "Combining federated learning and edge computing toward ubiquitous intelligence in 6g network: Challenges, recent advances, and future directions," *Commun. Surveys Tuts.*, vol. 25, no. 4, p. 2892–2950, Oct. 2023.

[9] G. Hukkeri, R. Goudar, D. Gm, V. Rathod, and S. Ankalaki, "Split-fed learning: A deep dive into methods, innovations and future prospects for data privacy and efficiency in decentralized machine learning," *IEEE Access*, vol. PP, pp. 1–1, 01 2025.

[10] M. Aggarwal, V. Khullar, and N. Goyal, "Fsl-tm: Review on the integration of federated split learning with tinyml in the internet of vehicles," *Computers, Materials & Continua*, vol. 86, pp. 1–31, 12 2025.

[11] C. Thapa, M. Chamikara, S. Camtepe, and L. Sun, "Splitfed: When federated learning meets split learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, pp. 8485–8493, 06 2022.

[12] L. Melis, C. Song, E. De Cristofaro, and V. Shmatikov, "Exploiting unintended feature leakage in collaborative learning," in *2019 IEEE symposium on security and privacy (SP)*. IEEE, 2019, pp. 691–706.

[13] A. Salem, Y. Zhang, M. Humbert, P. Berrang, M. Fritz, and M. Backes, "Ml-leaks: Model and data independent membership inference attacks and defenses on machine learning models," in *Proceedings of the 26th Annual Network and Distributed System Security Symposium (NDSS)*, 2019.

[14] L. Liu, Y. Wang, G. Liu, K. Peng, and C. Wang, "Membership inference attacks against machine learning models via prediction sensitivity," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 3, pp. 2341–2347, 2023.

[15] Z. Zhang, A. Pinto, V. Turina, F. Esposito, and I. Matta, "Privacy and efficiency of communications in federated split learning," *IEEE Transactions on Big Data*, vol. 9, no. 5, pp. 1380–1391, 2023.

[16] F. Wang, Y. Zhu, and B. Li, "Unraveling elevated data leakage in split learning for fine-tuning stable diffusion models," in *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, ser. ASIA CCS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 501–516.

[17] Z. He, T. Zhang, and R. B. Lee, "Attacking and protecting data privacy in edge–cloud collaborative inference systems," *IEEE Internet of Things Journal*, vol. 8, no. 12, pp. 9706–9716, 2021.

[18] D. Pasquini, G. Ateniese, and M. Bernaschi, "Unleashing the tiger: Inference attacks on split learning," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 2113–2129.

[19] Z. He, T. Zhang, and R. B. Lee, "Model inversion attacks against collaborative inference," in *Proceedings of the 35th Annual Computer Security Applications Conference*, ser. ACSAC '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 148–162.

[20] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds., vol. 32. Curran Associates, Inc., 2019.

[21] J. Deng, Y. Wang, J. Li, C. Wang, C. Shang, H. Liu, S. Rajasekaran, and C. Ding, "TAG: Gradient attack on transformer-based language models," in *Findings of the Association for Computational Linguistics: EMNLP 2021*, M.-F. Moens, X. Huang, L. Specia, and S. W.-t. Yih, Eds. Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 3600–3610.

[22] M. Balunović, D. I. Dimitrov, N. Jovanović, and M. Vechev, "Lamp: extracting text from gradients with language model priors," in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS '22. Red Hook, NY, USA: Curran Associates Inc., 2022.

[23] G. Chen, Z. Qin, M. Yang, Y. Zhou, T. Fan, T. Du, and Z. Xu, "Unveiling the vulnerability of private fine-tuning in split-based frameworks for large language models: A bidirectionally enhanced attack," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer*

and Communications Security, ser. CCS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 2904–2918.

[24] K. Chatzikokolakis, M. E. Andrés, N. E. Bordenabe, and C. Palamidessi, "Broadening the scope of differential privacy using metrics," in *Privacy Enhancing Technologies*, E. De Cristofaro and M. Wright, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 82–102.

[25] O. Feyisetan, B. Balle, T. Drake, and T. Diethe, "Privacy- and utility-preserving textual analysis via calibrated multivariate perturbations," in *Proceedings of the 13th International Conference on Web Search and Data Mining*, ser. WSDM '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 178–186.

[26] M. Du, X. Yue, S. S. M. Chow, T. Wang, C. Huang, and H. Sun, "Dp-forward: Fine-tuning and inference on language models with differential privacy in forward pass," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 2665–2679.

[27] M. Wu, G. Cheng, D. Ye, J. Kang, R. Yu, Y. Wu, and M. Pan, "Federated split learning with data and label privacy preservation in vehicular networks," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 1, pp. 1223–1238, 2024.

[28] T. Titcombe, A. J. Hall, P. Papadopoulos, and D. Romanini, "Practical defences against model inversion attacks for split neural networks," 2021.

[29] P. Vepakomma, A. Singh, O. Gupta, and R. Raskar, " NoPeek: Information leakage reduction to share activations in distributed deep learning ," in *2020 International Conference on Data Mining Workshops (ICDMW)*. Los Alamitos, CA, USA: IEEE Computer Society, Nov. 2020, pp. 933–942.

[30] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1175–1191.

[31] J. Bell, K. A. Bonawitz, A. Gascón, T. Lepoint, and M. Raykova, "Secure single-server aggregation with (poly)logarithmic overhead," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 1253–1269.

[32] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Theory of Cryptography*, ser. Lecture Notes in Computer Science, vol. 3876. Springer, 2006, pp. 265–284.

[33] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep learning with differential privacy," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 2016, pp. 308–318.

[34] R. C. Geyer, T. Klein, and M. Nabi, "Differentially private federated learning: A client level perspective," *arXiv preprint arXiv:1712.07557*, 2017.

[35] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang, "Learning differentially private recurrent language models," in *International Conference on Learning Representations*, 2018.

[36] N. D. Pham and N. Chilamkurti, "Data leakage threats and protection in split learning: A survey," in *Proceedings of the 2023 International Conference on Intelligent Computing and Its Emerging Applications*, ser. ICEA '23. New York, NY, USA: Association for Computing Machinery, 2024, p. 141–147.

[37] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," *arXiv preprint arXiv:1308.3432*, 2013.

[38] X. Li, K. Huang, W. Yang, S. Wang, and Z. Zhang, "On the convergence of fedavg on non-iid data," in *International Conference on Learning Representations*, 2020.

[39] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian *et al.*, "The llama 3 herd of models," 2024.

[40] A. H. Liu, K. Khandelwal, S. Subramanian, V. Jouault, A. Rastogi *et al.*, "Ministral 3," 2026.

[41] I. Casanueva, T. Temcinas, D. Gerz, M. Henderson, and I. Vulic, "Efficient intent detection with dual sentence encoders," in *Proceedings of the 2nd Workshop on NLP for ConvAI - ACL 2020*, mar 2020.

[42] S. Larson, A. Mahendran, J. J. Peper, C. Clarke, A. Lee, P. Hill, J. K. Kummerfeld, K. Leach, M. A. Laurenzano, L. Tang, and J. Mars, "An evaluation dataset for intent classification and out-of-scope prediction," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, K. Inui, J. Jiang, V. Ng, and X. Wan, Eds. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 1311–1316.

[43] J. Xu, T. Wei, B. Hou, P. Orzechowski, S. Yang, R. Jin, R. Paulbeck, J. Wagenaar, G. Demiris, and L. Shen, "Mentalchat16k: A benchmark dataset for conversational mental health assistance," 2025.

[44] A. Köpf, Y. Kilcher, D. von Rütte, S. Anagnostidis, Z. R. Tam, K. Stevens, A. Barhoum, D. Nguyen, O. Stanley, R. Nagyfi, S. ES, S. Suri, D. Glushkov, A. Dantuluri, A. Maguire, C. Schuhmann, H. Nguyen, and A. Mattick, "Openassistant conversations - democratizing large language model alignment," in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 47 669–47 681.

APPENDIX A
HYPERPARAMETERS

Table IV summarizes the hyperparameters used across all experiments. A few settings vary by model or dataset and are described here instead of in the table.

We set A to contain 5 transformer layers. Modern LLMs adopt a residual transformer architecture, and this gives the smashed data several layers of transformation beyond the raw input embeddings, so trivial reconstruction by attaching the language model head directly to s_A is not possible under this split. We set C to contain 1 layer.

We also vary the learning rate by dataset. Banking77 and CLINC150 are classification tasks in which the assistant response is a short label, so we set the learning rate to 1×10^{-5} for these two datasets. MentalChat16K requires generating a long, free-form assistant response, so we set the learning rate to 5×10^{-5} for this dataset.

When computing training effectiveness, we also vary the evaluation set size and the generation length by dataset. For Banking77 and CLINC150, we evaluate on 1,024 samples and generate at most 64 tokens per sample, matching the short label outputs of these classification tasks. For MentalChat16K, we evaluate on 256 samples and generate at most 256 tokens per sample, matching the longer free-form assistant responses of this dataset.

TABLE IV
HYPERPARAMETER SETTINGS USED IN ALL EXPERIMENTS.

Category	Parameter	Value
W_1 adapter	Layer num.	2
	Hidden dimension	$1 \times d_{\text{model}}$
W_2 adapter	Layer num.	2
	Hidden dimension	$2 \times d_{\text{model}}$
W_1 loss	$\lambda^{(1)}_{\text{CKA}}$	1.0
	$\lambda^{(1)}_{\text{JSD}}$	2.0
W_2 loss	$\lambda^{(2)}_{\text{KL}}$	1.0
	$\lambda^{(2)}_{\text{MSE}}$	2.0
Training	W_1 adapter learning rate	2×10^{-4}
	W_2 adapter learning rate	4×10^{-4}
	Adapter LR warmup steps	32
	Adapter LR eta min ratio	0.1
	Batch size	8
	Max sequence length	768
	Training steps	1,024
	Alignment steps	256
	Burst interval	256 training steps
	Burst alignment steps	16

$\lambda^{(1)}_{W_2}$ is also a coefficient in the W_1 loss, but its value is tuned separately for each model, as it depends on the model's response to W_1 obfuscation during alignment. Table V lists the value used for each model.

TABLE V
PER-MODEL VALUE OF $\lambda^{(1)}_{W_2}$.

Model	$\lambda^{(1)}_{W_2}$
Llama-3.2-3B	1.0
Llama-3.1-8B	0.6
Ministral-3-8B	1.4
Ministral-3-14B	1.4

Table VI lists the operating points used for the baselines on each model and dataset. ε is the Embedding ε-Privacy scale, σ is the Smashed-Data DP scale, and λ is the NoPeek scale.

TABLE VI
SELECTED OPERATING POINTS USED FOR THE BASELINES IN THE MAIN RESULTS.

Model	Dataset	ε	σ	λ
Llama-3.2-3B	Banking	0.1	0.2	1×10^2
	CLINC	0.1	0.2	5×10^1
	Mental	0.05	0.6	1×10^2
Llama-3.1-8B	Banking	0.2	0.2	5×10^1
	CLINC	0.2	0.2	1×10^2
	Mental	0.1	0.54	1×10^2
Ministral-3-8B	Banking	0.3	0.2	2.5×10^1
	CLINC	0.3	0.2	1×10^1
	Mental	0.2	0.6	1×10^2
Ministral-3-14B	Banking	0.2	0.2	1×10^1
	CLINC	0.2	0.2	1×10^1
	Mental	0.1	1	1×10^1

We implement TAG, LAMP, and BiSR following the official implementation of BiSR [23]. Direct Forward has no additional hyperparameters. All remaining attack hyperparameters follow the default settings of that implementation. Table VII lists these values.

TABLE VII
DEFAULT ATTACK HYPERPARAMETERS USED IN ALL EXPERIMENTS.

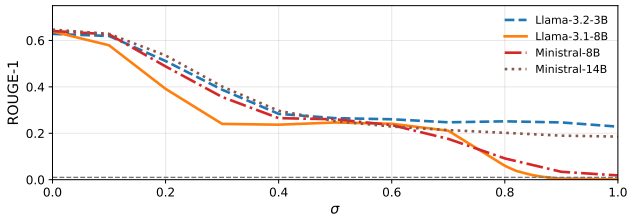
Category	Parameter	Value
TAG	Epochs	300
	β	0.85
	Learning rate	0.09
	Initialization temperature	1.0
LAMP	Epochs	300
	β	0.85
	Learning rate	0.09
	Reorder frequency	30
BiSR matching	Epochs	20
	Learning rate	1×10^{-4}
	Weight decay	0.01
BiSR inverter	Hidden size	256
	Dropout	0.1
	Learning rate	1×10^{-3}
	Weight decay	1×10^{-5}
	Epochs	20
	Batch size	6

The BiSR inverter is trained separately for each model and each downstream task. We reserve 512 samples from each dataset as an extra auxiliary dataset for this training. This auxiliary dataset does not overlap with any other split used in the experiments.

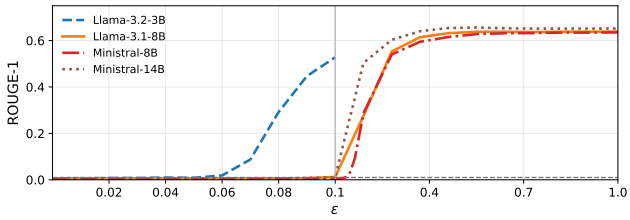
APPENDIX B
BASELINE HYPERPARAMETER TUNING

Each baseline exposes a hyperparameter that governs a privacy-utility trade-off, so a single fixed setting of this hyperparameter can make the baseline look arbitrarily weak along one axis while looking strong along the other. Comparing PrivPair against a baseline evaluated at only one such setting is therefore not sufficient to establish that PrivPair outperforms the baseline. To make this comparison meaningful, for each baseline we sweep its hyperparameter over a wide range and show that at some point in this range, the baseline is simultaneously no better than PrivPair on both privacy protection and training effectiveness. This rules out the possibility that the baseline's apparent weakness in our main results is an artifact of an unfavorable hyperparameter choice.

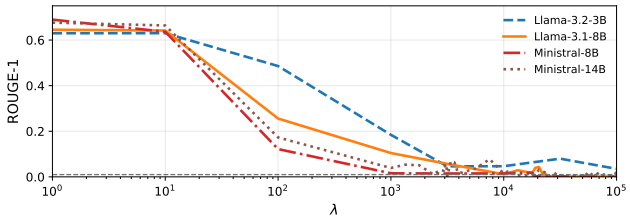
Figure 6 shows the resulting scan for each baseline. For Smashed-Data DP and Embedding ε-Privacy, we sweep a wide range of hyperparameter values directly on the base model and record the Direct Forward attack ROUGE-1 at each value. From this sweep we select three operating points. The first point is where the attack ROUGE-1 first drops noticeably, indicating that the defense has begun to take effect, and we treat this point as a lower bound on privacy. The second point is where the attack ROUGE-1 reaches 0.01, and we treat this point as an upper bound on privacy. Utility is not



(a) Smashed-Data DP



(b) Embedding ε-Privacy



(c) NoPeek

Fig. 6. Direct Forward attack ROUGE-1 on MentalChat16K across the scanned hyperparameter range for each baseline, shown for every model in the model suite. The onset, midpoint, and reach operating points described in this section are selected from these curves.

known at this stage, so these points are selected from the attack curve alone. The third point is the midpoint between the first and second points. For Embedding ε-Privacy and Smashed-Data DP this midpoint is the arithmetic mean. For NoPeek the midpoint is the rounded geometric mean. Because the NoPeek defense is applied as part of training and only becomes effective after training has proceeded for some time, we record the attack ROUGE-1 over the full training run and select three points from this curve using the same procedure. We carry out this tuning procedure on MentalChat16K and tune the hyperparameter separately for each model.

Tables VIII, IX, and X list the resulting onset, midpoint, and reach scales for each model. For some models the Direct Forward ROUGE-1 never falls to 0.01 within the scanned range, and in those cases reach is the strongest scale in the scan.

TABLE VIII
ONSET, MIDPOINT, AND REACH VALUES OF ε FOR EMBEDDING ε-PRIVACY ON MENTALCHAT16K.

Model	Onset	Midpoint	Reach
Llama-3.2-3B	0.1	0.075	0.05
Llama-3.1-8B	0.2	0.15	0.1
Ministral-3-8B	0.3	0.2	0.1
Ministral-3-14B	0.2	0.15	0.1

TABLE IX
ONSET, MIDPOINT, AND REACH VALUES OF σ FOR SMASHED-DATA DP ON MENTALCHAT16K.

Model	Onset	Midpoint	Reach
Llama-3.2-3B	0.2	0.6	1
Llama-3.1-8B	0.2	0.54	0.87
Ministral-3-8B	0.2	0.6	1
Ministral-3-14B	0.2	0.6	1

TABLE X
ONSET, MIDPOINT, AND REACH VALUES OF λ FOR NOPEEK ON MENTALCHAT16K.

Model	Onset	Midpoint	Reach
Llama-3.2-3B	1×10^2	3×10^3	1×10^5
Llama-3.1-8B	1×10^2	1×10^3	1×10^4
Ministral-3-8B	1×10^2	1×10^3	2×10^4
Ministral-3-14B	1×10^1	2×10^2	3×10^3

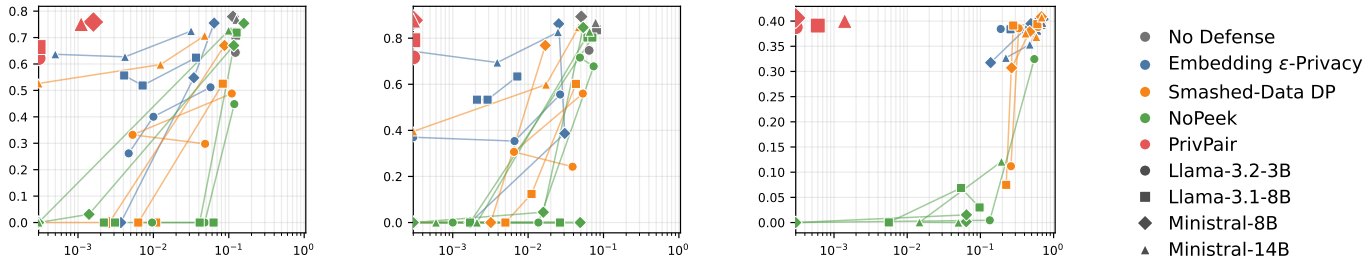
For NoPeek, the three operating points follow the same onset, midpoint, and reach rules as the other baselines. In practice, the midpoint and reach values of λ are often so large that training collapses completely. When all three scanned points fail in this way, we decrease λ and use a better scale as the selected operating point for that model and dataset.

Figure 7 places the scanned baseline scales, No Defense, and PrivPair in the privacy-utility plane. Utility is classification accuracy on Banking77 and CLINC150 and task ROUGE-1 on MentalChat16K. Direct Forward ROUGE-1 is measured at the final checkpoint. Color denotes the defense method, and marker shape denotes the model.

Tables XI, XII, and XIII report training effectiveness and Direct Forward leakage for every scanned operating point that is not the selected point used in the main results. Gray cells mark collapsed training, including near-zero task scores below 0.1. Light red cells mark points that are worse than PrivPair on both axes, with a lower task score and a higher Direct Forward score. A yellow task-score cell marks a non-collapsed point whose accuracy or task ROUGE-1 is below the corresponding base model. A light green task-score cell marks a point that is better than PrivPair on this axis.

APPENDIX C
DIRECT FORWARD ATTACK EXAMPLES

Figure 8 shows qualitative examples on Llama-3.2-3B. Each block shows the true label and compares three outputs for the



(a) Banking77 (b) CLINC150 (c) MentalChat16K

Fig. 7. Privacy-utility landscape of the scanned baseline scales, No Defense, and PrivPair. The horizontal axis is Direct Forward ROUGE-1 at the final checkpoint, shown on a log scale. The vertical axis is task utility. The top left is better, with lower leakage and higher utility.

TABLE XI
UNSELECTED EMBEDDING ε-PRIVACY OPERATING POINTS.

Model	Dataset	ε	Acc. / ROUGE-1	Direct Fwd ROUGE-1
Llama-3.2-3B	Banking	0.075	0.4004	0.0100
		0.05	0.2617	0.0047
	CLINC	0.075	0.3535	0.0066
		0.05	0.3701	0.0002
	Mental	0.1	0.3928	0.5721
		0.075	0.3797	0.4206
Llama-3.1-8B	Banking	0.15	0.5186	0.0072
		0.1	0.5566	0.0041
	CLINC	0.15	0.5332	0.0029
		0.1	0.5332	0.0021
	Mental	0.2	0.3841	0.5504
		0.15	0.3832	0.4655
Ministral-3-8B	Banking	0.2	0.5479	0.0344
		0.1	0.0000	0.0037
	CLINC	0.2	0.3867	0.0306
		0.1	0.0000	0.0017
	Mental	0.3	0.4041	0.6501
		0.1	0.3175	0.1380
Ministral-3-14B	Banking	0.15	0.6270	0.0042
		0.1	0.6367	0.0005
	CLINC	0.15	0.6943	0.0039
		0.1	0.7432	0.0000
	Mental	0.2	0.3809	0.6057
		0.15	0.3534	0.4728

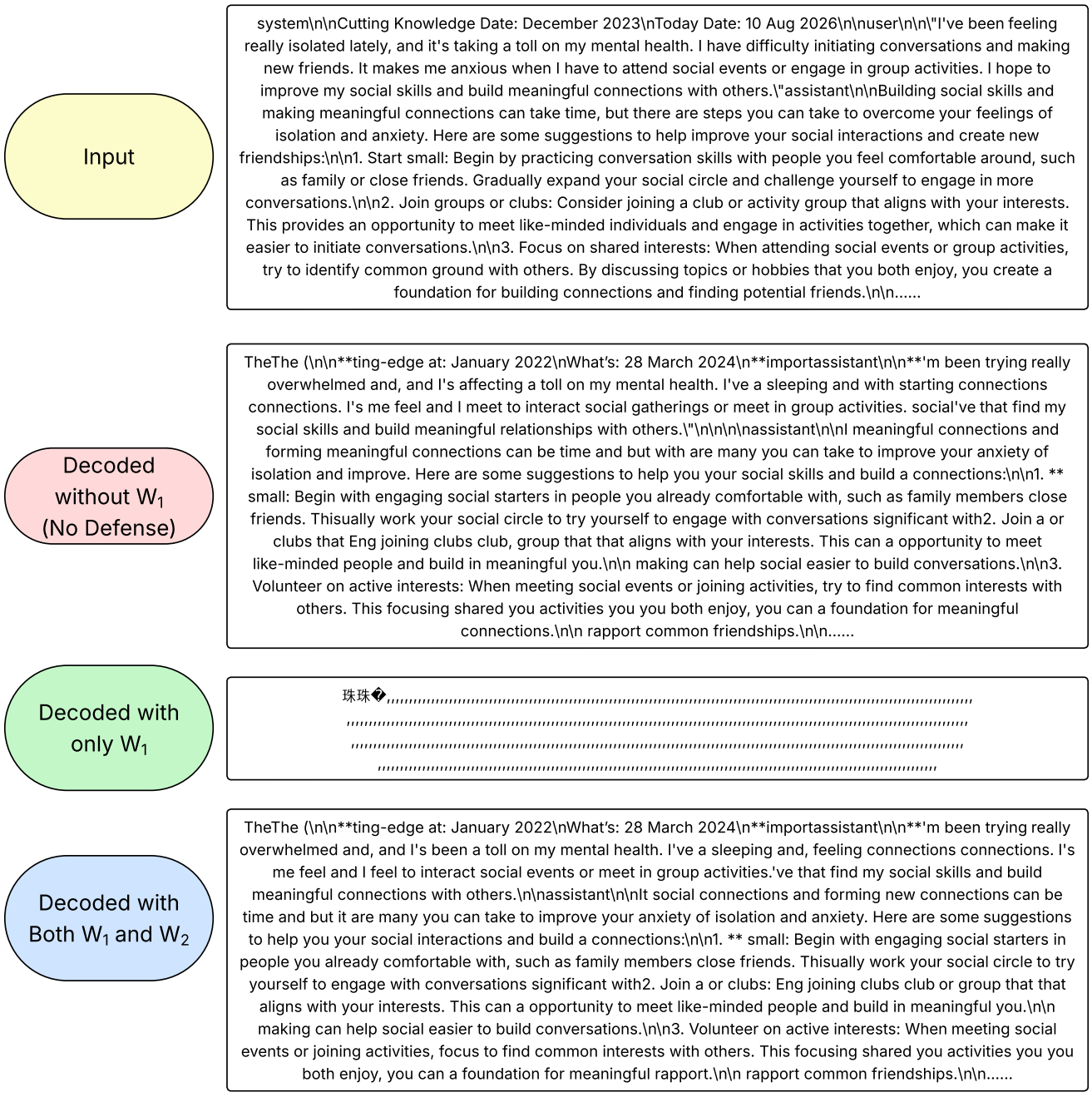
same input: the server's Direct Forward output without any defense, the server's Direct Forward output when only W_1 obfuscation is applied, and the client's output after the full $W_1 + W_2$ adapter protocol. Without defense, the server can extract meaningful information from the smashed data. With W_1 only, direct decoding by the server becomes uninformative because the obfuscated activations are incompatible with the server's decoder. With both W_1 and W_2, the client recovers substantially more coherent and task-relevant output from the returned activations while the server observes only the obfuscated path.

TABLE XII
UNSELECTED SMASHED-DATA DP OPERATING POINTS.

Model	Dataset	σ	Acc. / ROUGE-1	Direct Fwd ROUGE-1
Llama-3.2-3B	Banking	0.6	0.3320	0.0053
		1	0.2979	0.0485
	CLINC	0.6	0.3066	0.0065
		1	0.2422	0.0389
	Mental	0.2	0.3883	0.6130
		1	0.1119	0.2614
Llama-3.1-8B	Banking	0.54	0.0000	0.0063
		0.87	0.0000	0.0109
	CLINC	0.54	0.1240	0.0111
		0.87	0.0000	0.0050
	Mental	0.2	0.3940	0.5999
		0.87	0.0749	0.2251
Ministral-3-8B	Banking	0.6	0.0000	0.0026
		1	0.0000	0.0000
	CLINC	0.6	0.0000	0.0032
		1	0.0000	0.0000
	Mental	0.2	0.4087	0.6820
		1	0.3072	0.2670
Ministral-3-14B	Banking	0.6	0.5977	0.0124
		1	0.5264	0.0000
	CLINC	0.6	0.5986	0.0173
		1	0.3955	0.0000
	Mental	0.2	0.4073	0.6878
		0.6	0.3684	0.5811

TABLE XIII
UNSELECTED NOPEEK OPERATING POINTS.

Model	Dataset	λ	Acc. / ROUGE-1	Direct Fwd ROUGE-1
Llama-3.2-3B	Banking	3×10^3	0.0000	0.0479
		1×10^5	0.0000	0.0096
	CLINC	1×10^2	0.0000	0.0137
		3×10^3	0.0000	0.0010
		1×10^5	0.0000	0.0000
	Mental	3×10^3	0.0045	0.1343
		1×10^5	0.0000	0.0000
Llama-3.1-8B	Banking	2.5×10^1	0.7188	0.1268
		1×10^2	0.0000	0.0628
		1×10^3	0.0000	0.0031
		1×10^4	0.0000	0.0022
	CLINC	5×10^1	0.8027	0.0712
		1×10^3	0.0000	0.0265
		1×10^4	0.0000	0.0000
	Mental	1×10^3	0.0687	0.0544
		1×10^4	0.0003	0.0056
Ministral-3-8B	Banking	5×10^1	0.0312	0.0014
		1×10^2	0.0000	0.0000
		1×10^3	0.0000	0.0000
		2×10^4	0.0000	0.0000
	CLINC	2.5×10^1	0.0449	0.0159
		5×10^1	0.0000	0.0000
		1×10^2	0.0000	0.0491
		1×10^3	0.0000	0.0000
		2×10^4	0.0000	0.0000
	Mental	1×10^3	0.0153	0.0644
		2×10^4	0.0000	0.0000
Ministral-3-14B	Banking	2×10^2	0.0000	0.0000
		3×10^3	0.0000	0.0000
	CLINC	2×10^2	0.0000	0.0021
		3×10^3	0.0000	0.0006
	Mental	2×10^2	0.0005	0.0147
		3×10^3	0.0000	0.0500


Input
system\n\nCutting Knowledge Date: December 2023\nToday Date: 10 Aug 2026\n\nuser\n\n\"I've been feeling really isolated lately, and it's taking a toll on my mental health. I have difficulty initiating conversations and making new friends. It makes me anxious when I have to attend social events or engage in group activities. I hope to improve my social skills and build meaningful connections with others.\"assistant\n\nBuilding social skills and making meaningful connections can take time, but there are steps you can take to overcome your feelings of isolation and anxiety. Here are some suggestions to help improve your social interactions and create new friendships:\n\n1. Start small: Begin by practicing conversation skills with people you feel comfortable around, such as family or close friends. Gradually expand your social circle and challenge yourself to engage in more conversations.\n\n2. Join groups or clubs: Consider joining a club or activity group that aligns with your interests. This provides an opportunity to meet like-minded individuals and engage in activities together, which can make it easier to initiate conversations.\n\n3. Focus on shared interests: When attending social events or group activities, try to identify common ground with others. By discussing topics or hobbies that you both enjoy, you create a foundation for building connections and finding potential friends.\n\n.....
Decoded without W1 (No Defense)
TheThe (\n\n**ting-edge at: January 2022\nWhat's: 28 March 2024\n**importassistant\n\n**'m been trying really overwhelmed and, and I's affecting a toll on my mental health. I've a sleeping and with starting connections connections. I's me feel and I meet to interact social gatherings or meet in group activities. social've that find my social skills and build meaningful relationships with others.\"\n\n\n\nassistant\n\nI meaningful connections and forming meaningful connections can be time and but with are many you can take to improve your anxiety of isolation and improve. Here are some suggestions to help you your social skills and build a connections:\n\n1. ** small: Begin with engaging social starters in people you already comfortable with, such as family members close friends. Thisually work your social circle to try yourself to engage with conversations significant with2. Join a or clubs that Eng joining clubs club, group that that aligns with your interests. This can a opportunity to meet like-minded people and build in meaningful you.\n\n making can help social easier to build conversations.\n\n3. Volunteer on active interests: When meeting social events or joining activities, try to find common interests with others. This focusing shared you activities you you both enjoy, you can a foundation for meaningful connections.\n\n rapport common friendships.\n\n.....
Decoded with only W1
珠珠,,,